EE148b Assignment 4
Diffusion Models

May 16, 2024

Due: May 29, 2024 11:59pm

For Lateness and Regrading policies, check the syllabus

Please list below any resources that you used to do this homework or any person that helped you.

Jadelynn Dao

Please list your Colab links here:

ee148-hw4-Riti_Agarwal.ipynb

Learning Objectives

In this assignment, we dig more into Diffusion Modules (DM).

Learnings:

1. Go through the derivation of the Denoising Diffusion Probabilistic Models (DDPM)
2. Understand and compare multiple score matching techniques
3. Learn about the SDE perspective of diffusion models
4. Implement the training and sampling procedures of a diffusion model on a simple dataset
5. Further explore with a given codebase for more visualization and analysis

What and Where to Submit

You will submit the following files to Gradescope by **6/1/2023** at **2:00pm**:

- A PDF file `assignment4.pdf`, containing a **link** to your Colab notebook at the top of the file, requested figures, and answers to the questions asked in the assignment along with plots asked for in the assignment. Answers should be written clearly and succinctly; you may lose points if your answer is unclear or unnecessarily complex.
- To the **`HW4 - Code (Graded)`**
    - Your VP.py file.
      NOTE: We need to be able to run both your Colab notebook and your submitted code.

Getting the Code

Go to your Personal Google Drive account. **Copy** this Colab notebook into your Google Drive account. Download and unzip the .zip of starter/skeleton code from Piazza onto your local machine. We recommend first developing locally and then copying your code into the colab file.

Google Colab Pro+

You should have already set up the Google Colab Pro+ subscription in Assignment 2. If not, please refer to Assignment 2 for more information. To save compute units, please use the Premium GPU runtime only when you are actively running the experiments. Although Google automatically shut down your runtime if you are idling for a long time, it still takes your compute units before the runtime is shut down. You can change your runtime by going to `Runtime > Change runtime type > GPU class.`

Time Caution

This assignment consists of both math and coding problems. If you are unfamiliar with the technical details behind diffusion models, this assignment can take quite a lot of time, even though the runtime is not going to be very long. We strongly encourage **starting this assignment sooner rather than later** to allow sufficient time to train your models and get results.
Note: training each diffusion model in Part 4 takes ~10 minutes on an A100. Running code for Part 5 may take much longer.

Reminder for Project Proposal

The instruction and template for the project proposal are out. As indicated in the syllabus, the project proposal will be due on **June 2nd**. This is *one day after the deadline for this assignment*, so please **start early and plan your time accordingly**.

Part 1: DDPM: the iterative denoising perspective of diffusion models

(20pts)

In this part, we will delve deep into the (DDPM) paper [1] and understand the mathematical formulation behind it.

A. (0pt) Read through the paper [1]. Make sure you understand the main idea of the paper. The rest of this part will guide you through the derivations of DDPM. In later parts, we follow the notations in the paper.

B. (2pts) In the Background section of the paper, the authors wrote "the training [of diffusion models] is performed by optimizing the usual variational bound on negative log likelihood." Let's go into more details. Explain why we want to minimize the variational bound on negative log likelihood and prove the chain of (in)equalities in (3).

$$\mathbb{E}\left[-\log p_\theta(\mathbf{x}_0)\right] \leq \mathbb{E}_q\left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] = \mathbb{E}_q\left[-\log p(\mathbf{x}_T) - \sum_{t\geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})}\right] =: L \quad (3)$$

B. We want to prove:

$$E[-\log P_\theta(x_0)] \le E_q\left[-\log \frac{P_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] = E_q\left[-\log p(x_T) - \sum_{t\ge 1} \frac{P_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right] =: L$$

We know $q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$ from the forward process - it is a fixed Markov chain that gradually adds gaussian noise to the data according to a variance schedule: $\beta_1, \dots, \beta_T$

We also know that $q(x_t|x_{t-1}) := N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$

We know $P_\theta(x_{0:T}) := p(x_T)\prod_{t=1}^{T} P_\theta(x_{t-1}|x_t)$ is the reverse process, which is a Markov chain with learned Gaussian transitions starting at $p(x_T) = N(x_T; 0, I)$ (this is just pure noise)

We also know that $P_\theta(x_{t-1}|x_t) := N(x_{t-1}; \mu_\theta(x_t,t), \Sigma_\theta(x_t,t))$

The variational bound on the negative log likelihood is:

$$E[-\log P_\theta(x_0)] \le E_q\left[-\log \frac{P_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$$

$$\Rightarrow E[-\log P_\theta(x_0)] \le \underbrace{E_q[-\log P_\theta(x_{0:T}) + \log q(x_{1:T}|x_0)]}_{\text{let us expand this:}} \quad \text{since } \log\left(\frac{P_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right) = \log P_\theta(x_{0:T}) - \log q(x_{1:T}|x_0)$$

We know that $P_\theta(x_{0:T}) := p(x_T)\prod_{t=1}^{T} P_\theta(x_{t-1}|x_t)$ from the reverse process

$$-\log P_\theta(x_{0:T}) = -\log\left(p(x_T)\cdot\prod_{t=1}^{T} P_\theta(x_{t-1}|x_t)\right) = -\log p(x_T) - \log\left(\sum_{t=1}^{T} P_\theta(x_{t-1}|x_t)\right) = -\log p(x_T) - \sum_{t=1}^{T} \log P_\theta(x_{t-1}|x_t)$$

Next we know that $q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$ from the forward process

$$\log q(x_{1:T}|x_0) := \sum_{t=1}^{T} \log q(x_t|x_{t-1})$$

$$E_q[-\log P_\theta(x_{0:T}) + \log q(x_{1:T}|x_0)] = E_q[-\log p(x_T) - \sum_{t\ge 1}^{T} \log P_\theta(x_{t-1}|x_t) + \sum_{t=1}^{T} \log q(x_t|x_{t-1})]$$

Combining the summation term, we can say:

$$E_q[-\log p(x_T) - \sum_{t\ge 1}^{T} \log P_\theta(x_{t-1}|x_t) + \sum_{t=1}^{T} \log q(x_t|x_{t-1})] = E_q[-\log p(x_T) + \sum_{t=1}^{T} \log q(x_t|x_{t-1}) - \log P_\theta(x_{t-1}|x_t)]$$

$$= E_q\left[-\log p(x_T) + \sum_{t=1}^{T} \log \frac{q(x_t|x_{t-1})}{P_\theta(x_{t-1}|x_t)}\right] = E_q\left[-\log p(x_T) - \sum_{t=1}^{T} \log \frac{P_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right]$$

Let us define our loss function as:

$$L = E_q\left[-\log p(x_T) + \sum_{t\ge 1}^{T} (\log q(x_t|x_{t-1}) - \log P_\theta(x_{t-1}|x_t))\right] = E_q\left[-\log p(x_T) - \sum_{t=1}^{T} \log \frac{P_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right]$$

So, we have proved that: $E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] = E_q[-\log p_\theta(x_{0:T}) + \log q(x_{1:T}|x_0)] = E_q\left[-\log p(x_T) - \sum_{t=1}^{T} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right] := L$

Next, we just have to prove the variational Bound on Negative log likelihood, i.e we just want to prove $E\left[-\log p_\theta(x_0)\right] \leq E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$

To do this, we can use ELBO (Evidence lowerbound):

Deriving ELBO:

We know that minimising KL divergence is equal to maximising log-likelihood:

$E_{x\sim p^*(x)}[\ln p_\theta(x)] = -H(p^*) - D_{KL}(p^*(x)\|p_\theta(x))$, where $H(p^*) := -E_{x\sim p^*}[\log p^*(x)]$ is the entropy of the true distribution.

So, if we can maximise $E_{x\sim p^*(x)}[\ln p_\theta(x)]$ we minimise $D_{KL}(p^*(x)\|p_\theta(x))$ and can find the approximation $p_\theta \approx p^*$.

To maximise $E_{x\sim p^*(x)}[\log p_\theta(x)]$, we sample many $x_i \sim p^*(x)$, i.e use importance sampling:

$N \max_\theta E_{x\sim p^*(x)}[\ln p_\theta(x)] \approx \max_\theta \sum_i \log p_\theta(x_i)$, where N is the no. of samples drawn from the true distribution.

In order to maximise $\sum_i \ln p_\theta(x_i)$, we need to find $\ln p_\theta(x)$:

$$\ln p_\theta(x) = \ln \int p_\theta(x|z)\, p(z)\, dz$$

We can estimate this using Monte-Carlo integration w/ importance sampling:

$\int p_\theta(x|z)p(z)\,dz = E_{z\sim q_\phi(\cdot|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z|x)}\right]$ where $q_\phi(z|x)$ is a sampling distribution over z that we use to perform Monte-Carlo integration.

NOTE: If we sample $z \sim q_\phi(\cdot|x)$, then $\frac{p_\theta(x,z)}{q_\phi(z|x)}$ is an unbiased estimator of $p_\theta(x)$. But, since log is not linear, it is not an unbiased estimator of $\log p_\theta(x)$ and so we apply Jensen's inequality:

$$\log p_\theta(x) = \log E_{z\sim q_\phi(\cdot|x)}\left[\frac{p_\theta(x,z)}{q_\phi(z|x)}\right] \geq E_{z\sim q_\phi(\cdot|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z|x)}\right]$$

which gives us the ELBO inequality.

Now, we can use this to prove $E[-\log p_\theta(x_0)] \leq E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$

In our ELBO inequality, let $x = x_0$, $z = x_{1:T}$, $q_\phi = q$. This gives us: $\log p_\theta(x_0) \geq E_{x_{1:T}\sim q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)}\right]$

Taking the negative of both sides of the inequality: $-\log p_\theta(x_0) \leq -E_{x_{1:T}\sim q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_0, x_{1:T})}{q(x_{1:T}|x_0)}\right]$

We know joint distribution $p_\theta(x_0, x_{1:T})$ can be decomposed as: $p_\theta(x_0, x_{1:T}) = p_\theta(x_{0:T})$

So, the inequality becomes:

$$-\log p_\theta(x_0) \leq -E_{x_{1:T}\sim q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right] \quad -①$$

This means that

$$E[-\log p_\theta(x_0)] \leq E\left[-E_{x_{1:T}\sim q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]\right]$$

since ① holds for all x_i's.

So, we can write this as:

$$E[-\log p_\theta(x_0)] \leq E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$$

Thus we have proved every inequality (q equality) in eqn ③ in the question.

C. (4pts) The construction of a tractable loss function is one of the most important parts of the original diffusion model paper [2] and DDPM. The derivation is given in Appendix A of [1], but please provide more mathematical details to justify the why Equations (18), (20), (21), and (22) hold. These quantities involve simplifying terms, invoking established equalities/theorems, and using definitions. Please write out what equalities/theorems/definitions are being used for each of those steps.

C. We need to first justify $L = E_q\left[-\log p(x_T) - \sum_{t\geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right]$ — (18)

We have that $L = E_q\left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}\right]$ — (17)

We know $q(x_{1:T}|x_0) := \prod_{t=1}^{T} q(x_t|x_{t-1})$ from the forward process and we know $p_\theta(x_{0:T}) := p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$ is the reverse process

Substituting this into 17 we get:

$$L = E_q\left[-\log \frac{p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)}{\prod_{t=1}^{T} q(x_t|x_{t-1})}\right]$$

$$= E_q\left[-\log p(x_T) - \sum_{t=1}^{T} \log p_\theta(x_{t-1}|x_t) + \sum_{t=1}^{T} \log q(x_t|x_{t-1})\right]$$

$$= E_q\left[-\log p(x_T) - \left(\sum_{t\geq 1} \log p_\theta(x_{t-1}|x_t) - \log q(x_t|x_{t-1})\right)\right]$$

$$= E_q\left[-\log p(x_T) - \sum_{t\geq 1} \log \frac{p(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right]$$

Next we need to justify eqn 20: $L = E_q\left[-\log p(x_T) - \sum_{t\geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t,x_0)} \cdot \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} - \log \frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\right]$

given eqn. 18: $L = E_q\left[-\log p(x_T) - \sum_{t\geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\right]$

In eqn 18, we can split the sum into separate terms for when $t>1$ and $t=1$:

$$\sum_{t\geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} = \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} + \log \frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}$$

↳ sum when $t>1$ ↳ term when $t=1$

Substituting this back into eqn 18, we get:

$$L = E_q\left[-\log p(x_T) - \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} - \log \frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\right] \quad — (19)$$

Next, we can write $q(x_t|x_{t-1})$ as $q(x_t|x_{t-1}) = \frac{q(x_t|x_{t-1},x_0)\cdot q(x_{t-1}|x_0)}{q(x_t|x_0)}$. Let us prove this:

Bayes' Theorem states that $P(A|B) = \frac{P(B|A)\cdot P(A)}{P(B)}$

Applying this to $q(x_t|x_{t-1})$ we get $q(x_t|x_{t-1}) = \frac{q(x_{t-1}|x_t)\cdot q(x_t)}{q(x_{t-1})}$. Here, all of the terms have really vast variances since this is not for our specific input image, this is for all possible input images. We have to narrow this down so we can condition the eqn on our input image. i.e. condition the eqn on x_0:

$$q(x_t|x_{t-1}) = \frac{q(x_{t-1}|x_t)\cdot q(x_t)}{q(x_{t-1})}\Big|\, x_0 \;\Rightarrow\; \frac{q(x_{t-1}|x_t, x_0)\cdot q(x_t|x_0)}{q(x_{t-1}|x_0)}$$

Now, we can substitute $q(x_t|x_{t-1}) = \frac{q(x_t|x_{t-1}, x_0)\cdot q(x_{t-1}|x_0)}{q(x_t|x_0)}$ into (19) to get:

$$L = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big] = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)\cdot q(x_{t-1}|x_0)}{q(x_{t-1}|x_t, x_0)\cdot q(x_t|x_0)} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big]$$

So, we have proved eqn 20:

$$L = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big] \quad (20)$$

Finally, we want to show $L = E_q\big[D_{KL}(q(x_T|x_0)\,\|\,p(x_T)) + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t, x_0)\,\|\,p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)\big]$

given $L = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big]$:

$$L = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\left(\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}\right) - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big]$$

We can express $\sum_{t\geq 1}\log\left(\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}\right) = \sum_{t>1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \sum_{t>1}\log\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}$

Now, let us specifically look at $\sum_{t\geq 1}\log\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}$. We can simplify this to $\log\frac{q(x_1|x_0)}{q(x_T|x_0)}$. Let us illustrate this through an example. Let $T=4$:

then, $\sum_{t\geq 1}\log\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} = \log\left(\prod_{t=2}^{4}\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}\right) = \log\left(\frac{q(x_1|x_0)}{q(x_2|x_0)}\cdot\frac{q(x_2|x_0)}{q(x_3|x_0)}\cdot\frac{q(x_3|x_0)}{q(x_4|x_0)}\right)$. We can see that only the last denominator term & the first numerator term remains & everything else cancels out, leaving us with $\log\frac{q(x_1|x_0)}{q(x_0|x_0)}$. We can clearly see that this can be applied to any value of T, so we can write: $\sum_{t\geq 1}\log\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} = \log\frac{q(x_1|x_0)}{q(x_T|x_0)}$. So now we have:

$$\sum_{t\geq 1}\log\left(\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}\right) = \sum_{t>1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \sum_{t>1}\log\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} = \sum_{t>1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \log\frac{q(x_1|x_0)}{q(x_T|x_0)}.$$

Substituting this back into eqn (20) we get:

$$L = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\left(\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}\cdot\frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}\right) - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big] = E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \log\frac{q(x_1|x_0)}{q(x_T|x_0)} - \log\frac{p_\theta(x_0|x_1)}{q(x_1|x_0)}\Big]$$

$$= E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \cancel{\log q(x_1|x_0)} + \log q(x_T|x_0) - \log p_\theta(x_0|x_1) + \cancel{\log q(x_1|x_0)}\Big]$$

$$= E_q\Big[-\log p(x_T) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \log q(x_T|x_0) - \log p_\theta(x_0|x_1)\Big] = E_q\Big[-(\log p(x_T) - \log q(x_T|x_0)) - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \log p_\theta(x_0|x_1)\Big]$$

↓ Bring this to the front

$$= E_q\Big[-\log\frac{p(x_T)}{q(x_T|x_0)} - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \log p_\theta(x_0|x_1)\Big] \quad (21)$$

We know that the KL divergence is defined as:

$$D_{KL}(p\|q) = E_p\left[\log\frac{p(x)}{q(x)}\right] = E_p[\log p(x) - \log q(x)]$$

We can rewrite eqn 21 as:

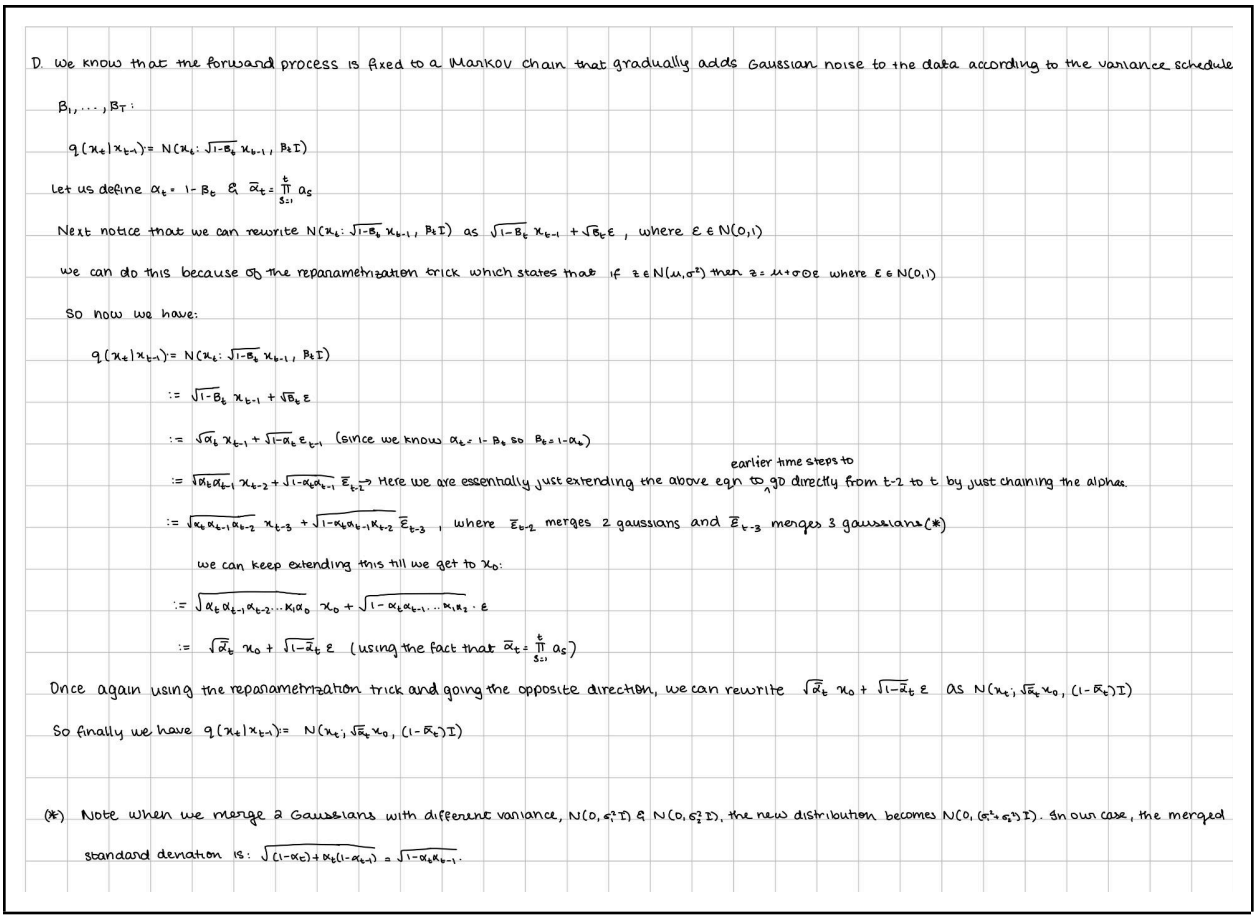
$$L = E_q\Big[-\log\frac{p(x_T)}{q(x_T|x_0)} - \sum_{t\geq 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} - \log p_\theta(x_0|x_1)\Big] = E_q\Big[\log\frac{q(x_T|x_0)}{p(x_T)} + \sum_{t\geq 1}\log\frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} - \log p_\theta(x_0|x_1)\Big]$$

We can rewrite this in terms of KL divergence:

$$L = E_q\big[D_{KL}(q(x_T|x_0)\,\|\,p(x_T)) + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t, x_0)\,\|\,p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)\big]$$

D. (2pts) The derivation of B and C is general, but now let's focus on the Markov process of the DDPM. First prove Equation (4):

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}) \tag{4}$$



D. We know that the forward process is fixed to a Markov chain that gradually adds Gaussian noise to the data according to the variance schedule $\beta_1, \dots, \beta_T$:

$q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$

Let us define $\alpha_t = 1-\beta_t$ & $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$

Next notice that we can rewrite $N(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$ as $\sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\varepsilon$, where $\varepsilon \in N(0,1)$

we can do this because of the reparametrization trick which states that if $z \in N(\mu, \sigma^2)$ then $z = \mu + \sigma\varepsilon$ where $\varepsilon \in N(0,1)$

So now we have:

$q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$

$:= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\varepsilon$

$:= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}$ (since we know $\alpha_t = 1-\beta_t$ so $\beta_t = 1-\alpha_t$)

$:= \sqrt{\alpha_t\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\, \bar{\varepsilon}_{t-2}$ → here we are essentially just extending the above eqn to go directly from t-2 to t by just chaining the alphas. earlier time steps to

$:= \sqrt{\alpha_t\alpha_{t-1}\alpha_{t-2}}\, x_{t-3} + \sqrt{1-\alpha_t\alpha_{t-1}\alpha_{t-2}}\, \bar{\varepsilon}_{t-3}$, where $\bar{\varepsilon}_{t-2}$ merges 2 gaussians and $\bar{\varepsilon}_{t-3}$ merges 3 gaussians (*)

we can keep extending this till we get to x_0:

$:= \sqrt{\alpha_t\alpha_{t-1}\alpha_{t-2}\cdots\alpha_1\alpha_0}\, x_0 + \sqrt{1-\alpha_t\alpha_{t-1}\cdots\alpha_1\alpha_0}\cdot\varepsilon$

$:= \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ (using the fact that $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$)

Once again using the reparametrization trick and going the opposite direction, we can rewrite $\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ as $N(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t)I)$

So finally we have $q(x_t|x_{t-1}) = N(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t)I)$

(*) Note when we merge 2 Gaussians with different variance, $N(0, \sigma_1^2 I)$ & $N(0, \sigma_2^2 I)$, the new distribution becomes $N(0, (\sigma_1^2 + \sigma_2^2)I)$. In our case, the merged standard deviation is: $\sqrt{(1-\alpha_t) + \alpha_t(1-\alpha_{t-1})} = \sqrt{1-\alpha_t\alpha_{t-1}}$.

E. (6pts) Then prove Equation (6) and (7):

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbf{I}), \tag{6}$$

$$\text{where} \quad \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t \quad \text{and} \quad \tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t \tag{7}$$

Hint: Bayes' rule is one way to do it, but there is a simpler way. Consider the joint distribution of x_{t-1} and x_t conditioned on x_0, and then use the Gaussian conditioning formula. Try using the reparametrization trick to write x_t in terms of x_{t-1}. This list of covariance properties can be useful: https://en.wikipedia.org/wiki/Covariance#Properties.

E. FROM bayes Rule:

$$q(x_{t-1}, x_t \mid x_0) = \frac{q(x_t \mid x_{t-1}, x_0) \cdot q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}$$

$$= \frac{\mathcal{N}(x_t; \sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t)\mathbf{I}) \cdot \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, x_0, (1-\bar{\alpha}_{t-1})\mathbf{I})}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_{t-1})\mathbf{I})}$$

$$\propto \exp\left[-\frac{1}{2}\left(\frac{(x_t - \sqrt{\alpha_t}\, x_{t-1})^2}{1-\alpha_t} + \frac{(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\, x_0)^2}{1-\bar{\alpha}_{t-1}} - \frac{(x_t - \sqrt{\bar{\alpha}_t}\, x_0)^2}{1-\bar{\alpha}_t}\right)\right]$$

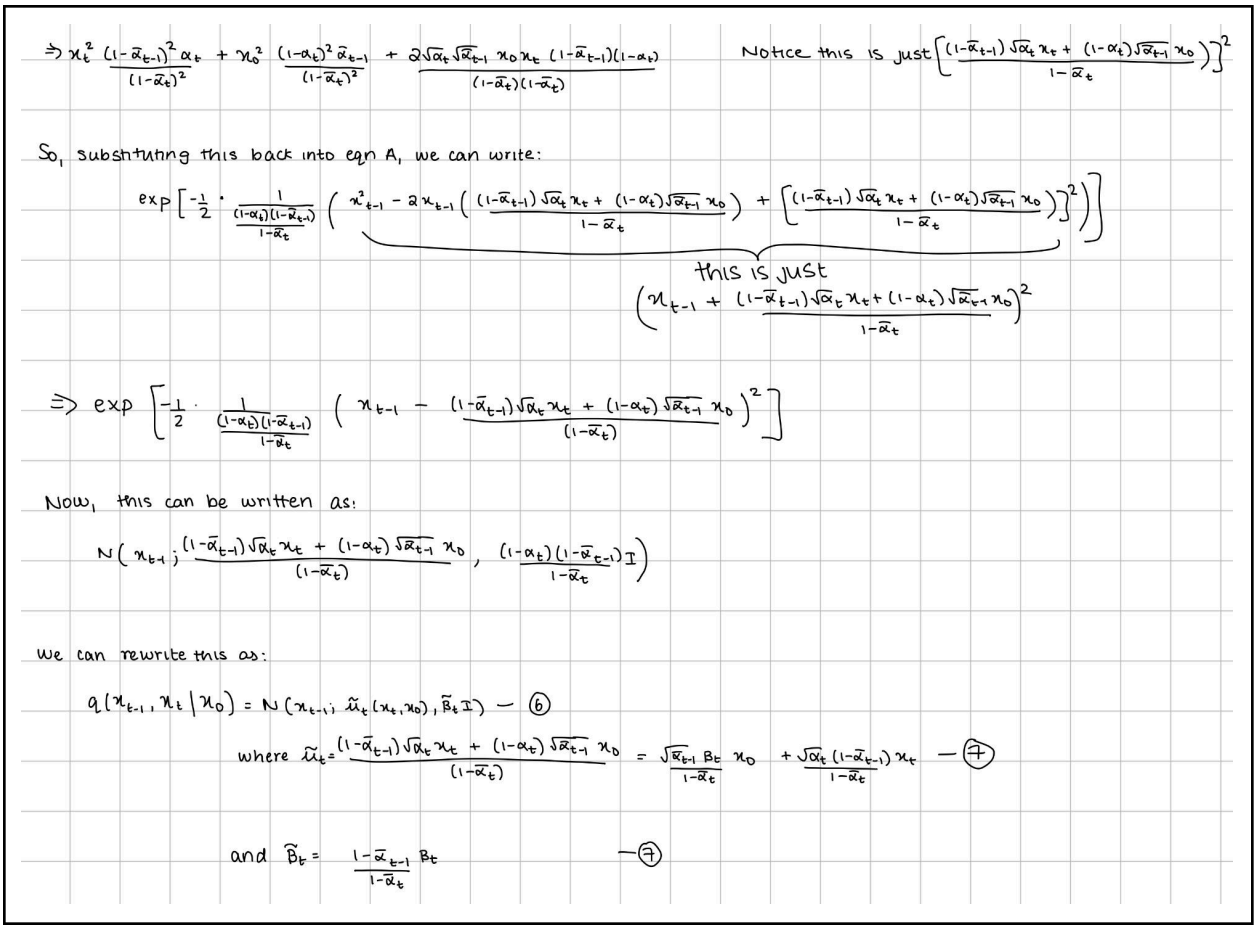
$$= \exp\left[-\frac{1}{2}\left(x_{t-1}^2\left(\frac{\alpha_t}{1-\alpha_t} + \frac{1}{1-\bar{\alpha}_{t-1}}\right) - 2x_{t-1}\left(\frac{\sqrt{\alpha_t}\, x_t}{1-\alpha_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}\, x_0}{1-\bar{\alpha}_{t-1}}\right) + (\cdots)\right)\right]$$

$$= \exp\left[-\frac{1}{2}\left(x_{t-1}^2\left(\frac{\alpha_t - \bar{\alpha}_t + 1 - \alpha_t}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}\right) - 2x_{t-1}\left(\frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}\, x_t + (1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}\, x_0}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}\right) + (\cdots)\right)\right]$$

$$= \exp\left[-\frac{1}{2} \cdot \frac{1}{\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}}\left(x_{t-1}^2 - 2x_{t-1}\left(\frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}\, x_t + (1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}\, x_0}{1-\bar{\alpha}_t}\right) + \left(\frac{1-\alpha_t}{1-\bar{\alpha}_t}\right)(\cdots)\right)\right] \quad \text{— (A)}$$

$$\frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}(\cdots) = \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\left(\frac{x_t^2}{1-\alpha_t} + \frac{\bar{\alpha}_{t-1}^2\, x_0^2}{1-\bar{\alpha}_{t-1}} - \frac{x_t^2}{1-\bar{\alpha}_t} - \frac{\bar{\alpha}_t x_0^2}{1-\bar{\alpha}_t} + \frac{2\sqrt{\bar{\alpha}_t}\, x_0 x_t}{1-\bar{\alpha}_t}\right)$$

$$\Rightarrow x_t^2\,\frac{(1-\bar{\alpha}_{t-1})(1-\bar{\alpha}_t) - (1-\bar{\alpha}_{t-1})(1-\alpha_t)}{(1-\bar{\alpha}_t)(1-\bar{\alpha}_t)} + x_0^2\,(1-\bar{\alpha}_t)(1-\alpha_t)(\bar{\alpha}_{t-1}) - \frac{(1-\bar{\alpha}_{t-1})(1-\bar{\alpha}_{t-1})(1-\alpha_t)\bar{\alpha}_t}{(1-\bar{\alpha}_t)(1-\bar{\alpha}_t)} + \frac{2\sqrt{\bar{\alpha}_t}\, x_0 x_t\,(1-\bar{\alpha}_{t-1})(1-\alpha_t)}{(1-\bar{\alpha}_t)(1-\bar{\alpha}_t)}$$



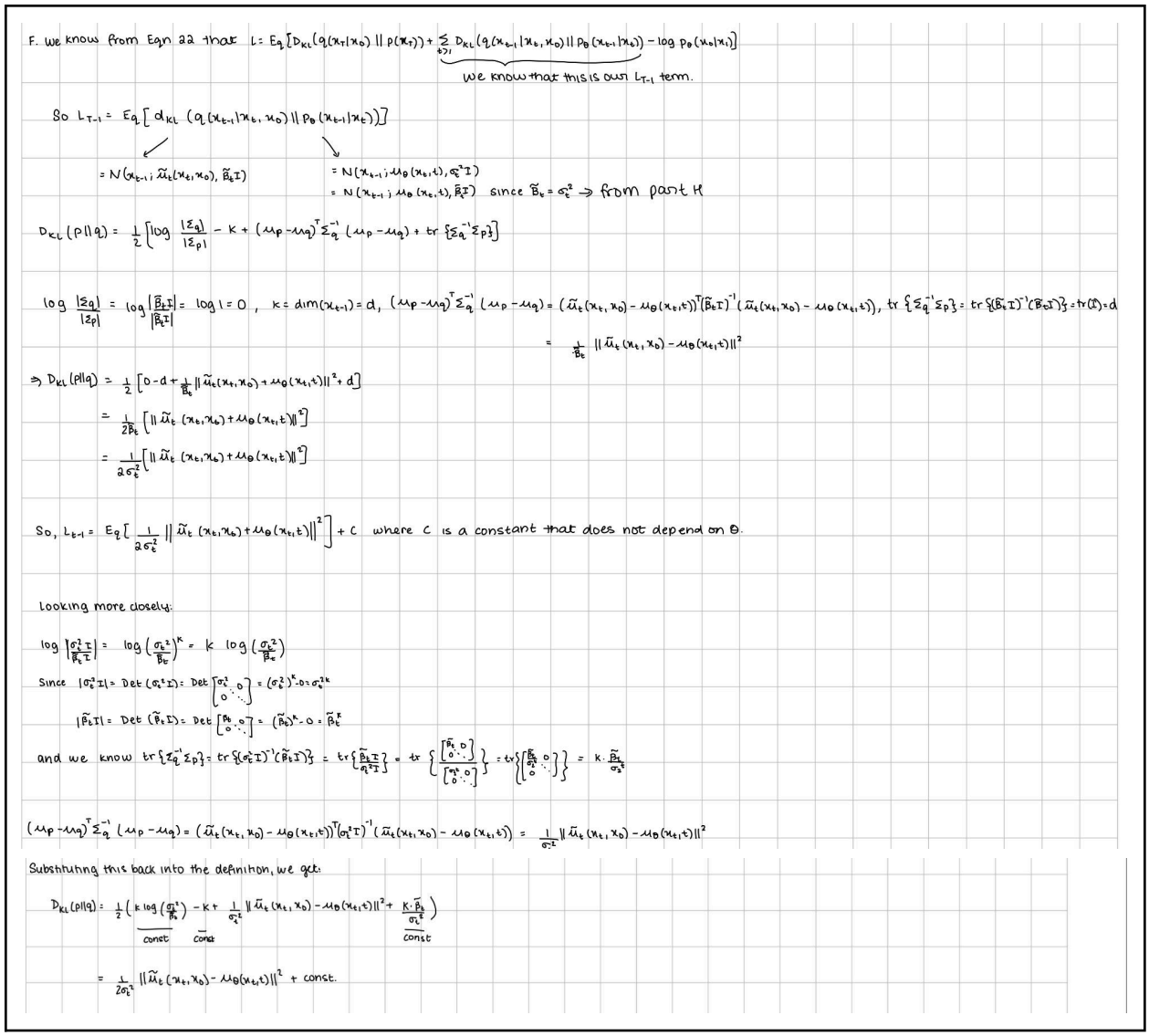
F. (2pts) The KL divergence between two k-dimensional multivariate Gaussian distributions $p(\mu_p, \Sigma_p)$ and $q(\mu_q, \Sigma_q)$ can be computed in closed form as follows:

$$D_{KL}(p||q) = \frac{1}{2}\left[\log\frac{|\Sigma_q|}{|\Sigma_p|} - k + (\boldsymbol{\mu_p} - \boldsymbol{\mu_q})^T\Sigma_q^{-1}(\boldsymbol{\mu_p} - \boldsymbol{\mu_q}) + tr\left\{\Sigma_q^{-1}\Sigma_p\right\}\right]$$

Assume that $\Sigma_\theta(x_t, t) = \tilde{\beta}_t I$ where $\tilde{\beta}_t$ is defined in Equation (7). Use the above fact to prove Equation (8):

$$L_{t-1} = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}\|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2\right] + C \tag{8}$$

where C is a constant that does not depend θ.



F. We know from Eqn 22 that $L = E_q[D_{KL}(q(x_T|x_0) \| p(x_T)) + \sum_{t>1} D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)]$

we know that this is our L_{T-1} term.

So $L_{T-1} = E_q[D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t))]$

$= N(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$

$= N(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$

$= N(x_{t-1}; \mu_\theta(x_t, t), \tilde{\beta}_t I)$ since $\tilde{\beta}_t = \sigma_t^2 \Rightarrow$ from past HW

$D_{KL}(p\|q) = \frac{1}{2}\left[\log \frac{|\Sigma_q|}{|\Sigma_p|} - k + (\mu_p - \mu_q)^T \Sigma_q^{-1} (\mu_p - \mu_q) + \text{tr}\{\Sigma_q^{-1}\Sigma_p\}\right]$

$\log \frac{|\Sigma_q|}{|\Sigma_p|} = \log \frac{|\tilde{\beta}_t I|}{|\tilde{\beta}_t I|} = \log 1 = 0$, $k = \dim(x_{t-1}) = d$, $(\mu_p - \mu_q)^T \Sigma_q^{-1} (\mu_p - \mu_q) = (\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t))^T (\tilde{\beta}_t I)^{-1} (\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t))$, $\text{tr}\{\Sigma_q^{-1}\Sigma_p\} = \text{tr}\{(\tilde{\beta}_t I)^{-1}(\tilde{\beta}_t I)\} = \text{tr}(I) = d$

$= \frac{1}{\tilde{\beta}_t} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2$

$\Rightarrow D_{KL}(p\|q) = \frac{1}{2}\left[0 - d + \frac{1}{\tilde{\beta}_t}\|\tilde{\mu}_t(x_t, x_0) + \mu_\theta(x_t, t)\|^2 + d\right]$

$= \frac{1}{2\tilde{\beta}_t}\left[\|\tilde{\mu}_t(x_t, x_0) + \mu_\theta(x_t, t)\|^2\right]$

$= \frac{1}{2\sigma_t^2}\left[\|\tilde{\mu}_t(x_t, x_0) + \mu_\theta(x_t, t)\|^2\right]$

So, $L_{t-1} = E_q\left[\frac{1}{2\sigma_t^2}\left\|\tilde{\mu}_t(x_t, x_0) + \mu_\theta(x_t, t)\right\|^2\right] + C$ where C is a constant that does not depend on θ.

Looking more closely:

$\log \frac{|\sigma_t^2 I|}{|\tilde{\beta}_t I|} = \log\left(\frac{\sigma_t^2}{\tilde{\beta}_t}\right)^k = k \log\left(\frac{\sigma_t^2}{\tilde{\beta}_t}\right)$

Since $|\sigma_t^2 I| = \text{Det}(\sigma_t^2 I) = \text{Det}\begin{bmatrix}\sigma_t^2 & 0 \\ 0 & \ddots\end{bmatrix} = (\sigma_t^2)^k - 0 = \sigma_t^{2k}$

$|\tilde{\beta}_t I| = \text{Det}(\tilde{\beta}_t I) = \text{Det}\begin{bmatrix}\tilde{\beta}_t & 0 \\ 0 & \ddots\end{bmatrix} = (\tilde{\beta}_t)^k - 0 = \tilde{\beta}_t^k$

and we know $\text{tr}\{\Sigma_q^{-1}\Sigma_p\} = \text{tr}\{(\sigma_t^2 I)^{-1}(\tilde{\beta}_t I)\} = \text{tr}\left\{\frac{\tilde{\beta}_t I}{\sigma_t^2 I}\right\} = \text{tr}\left\{\begin{bmatrix}\frac{\tilde{\beta}_t}{\sigma_t^2} & 0 \\ 0 & \ddots\end{bmatrix}\right\} = \text{tr}\left\{\begin{bmatrix}\frac{\tilde{\beta}_t}{\sigma_t^2} & 0 \\ 0 & \ddots\end{bmatrix}\right\} = k \cdot \frac{\tilde{\beta}_t}{\sigma_t^2}$

$(\mu_p - \mu_q)^T \Sigma_q^{-1} (\mu_p - \mu_q) = (\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t))^T (\sigma_t^2 I)^{-1} (\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)) = \frac{1}{\sigma_t^2}\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2$

Substituting this back into the definition, we get:

$D_{KL}(p\|q) = \frac{1}{2}\left(\underbrace{k \log\left(\frac{\sigma_t^2}{\tilde{\beta}_t}\right)}_{\text{const}} - \underbrace{k}_{\text{const}} + \frac{1}{\sigma_t^2}\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 + \underbrace{\frac{k \cdot \tilde{\beta}_t}{\sigma_t^2}}_{\text{const}}\right)$

$= \frac{1}{2\sigma_t^2}\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 + \text{const.}$

G. (2pts) Plug the parameterization in Equation (11) into Equation (10) and show that $L_{t-1} - C$ is indeed equal to (12):

$$\mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t)\right\|^2\right] \tag{12}$$

G. $\mu_\theta(x_t,t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t,t)\right)$ — from eqn 11

$L_{t-1} - C = E_{x_0,\varepsilon}\left[\frac{1}{2\sigma_t^2}\left\|\frac{1}{\sqrt{\alpha_t}}\left(x_t(x_0,\varepsilon) - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right) - \mu_\theta(x_t(x_0,\varepsilon),t)\right\|^2\right]$ — eqn 10

Substituting eqn 11 into eqn 10 we get:

$L_{t-1} - C = E_{x_0,\varepsilon}\left[\frac{1}{2\sigma_t^2}\left\|\frac{1}{\sqrt{\alpha_t}}\left(x_t(x_0,\varepsilon) - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right) - \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t,t)\right)\right\|^2\right]$

$\Rightarrow E_{x_0,\varepsilon}\left[\frac{1}{2\sigma_t^2}\left\|\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon - x_t + \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t,t)\right)\right\|^2\right]$

$\Rightarrow E_{x_0,\varepsilon}\left[\frac{1}{2\sigma_t^2}\left\|\frac{1}{\sqrt{\alpha_t}}\cdot\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\left(-\varepsilon + \varepsilon_\theta(x_t,t)\right)\right\|^2\right] = E_{x_0,\varepsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|\left(-\varepsilon + \varepsilon_\theta(x_t,t)\right)\right\|^2\right]$

$L_{t-1} - C \Rightarrow E_{x_0,\varepsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|(\varepsilon - \varepsilon_\theta(x_t,t))\right\|^2\right]$

Lastly, we know that $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ where $\varepsilon \in N(0,I)$ since $q(x_t|x_0) = N(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$, and so by the reparameterization trick that we have defined above, we get: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$ where $\varepsilon \in N(0,I)$

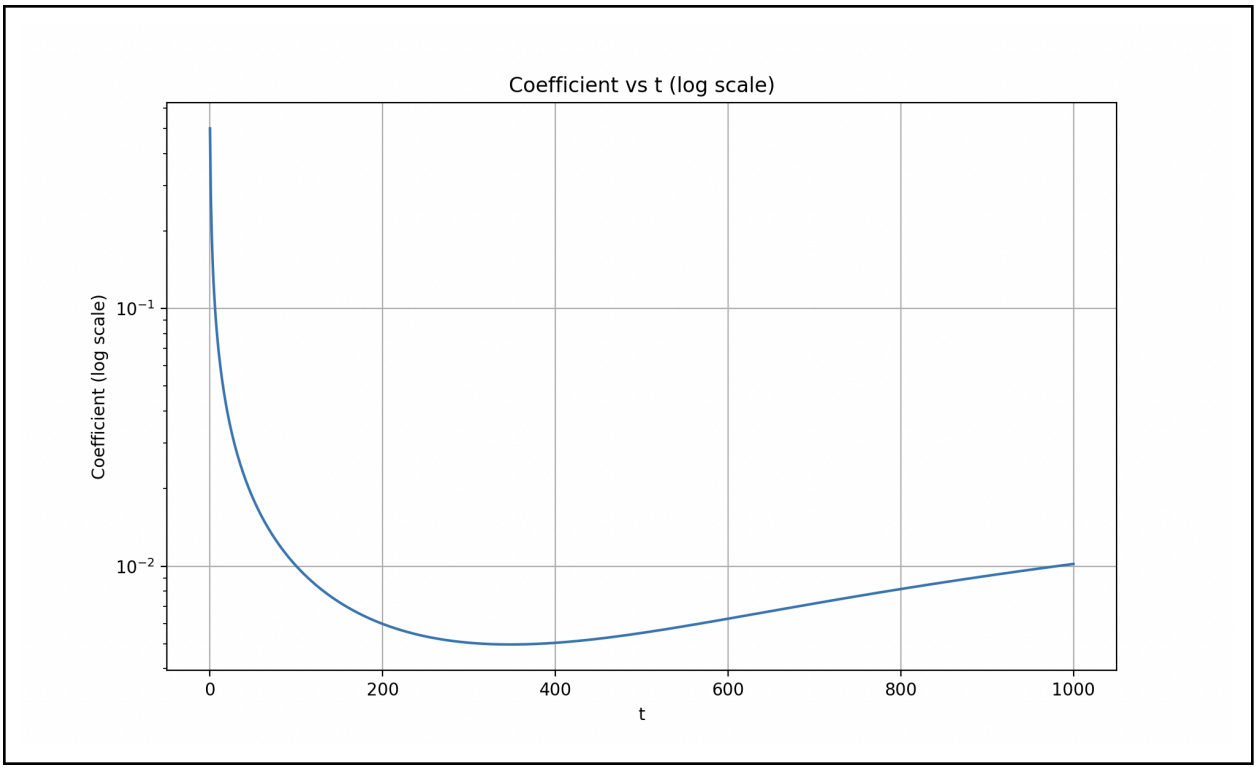
Substituting this into our eqn for $L_{t-1} - C$, we get:

$L_{t-1} - C = E_{x_0,\varepsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|(\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon, t))\right\|^2\right]$

H. (2pts) Assume β_t defined in Section 4 and $\sigma_t^2 = \tilde{\beta}_t$ defined in Equation (7). Plot the coefficient (**y axis in log scale**)

$$\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$$

with respect to t (x axis). Note how the coefficient is relatively smaller for large t. This observation means that the simplified objective, which drops the coefficients, is equivalent to down-weight the loss terms for smaller t. Empirically, this leads to better performance. See a discussion in Section 3.4 of the paper.



Part 1: Total 20/20 (20/85 + 15EC)

Part 2: Score matching and Langevin dynamics

(20pts)

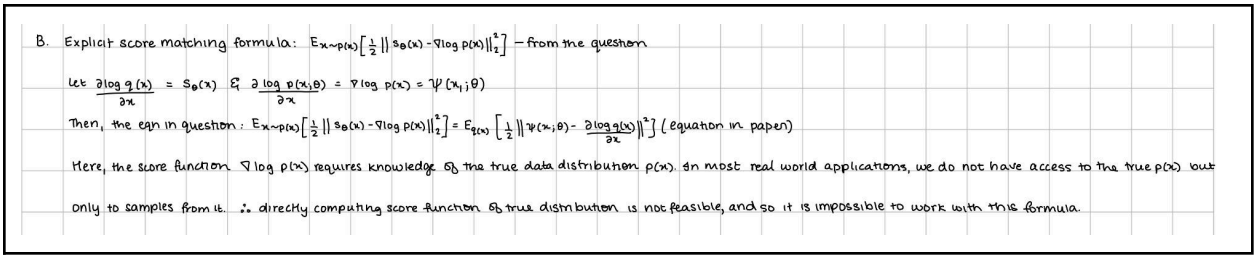
In this part, we are going to shift our perspective from denoising to score function. By definition, the (Stein) score function of a probability distribution with density $p(x)$ is $\nabla \log p(x)$ where the gradient is taken with respect to x.

A. (0pt) Read through the paper [3]. Make sure you understand the main idea of the paper. In the following parts, we will learn more about some of the score matching techniques mentioned in the paper.

B. (1pt) In score matching, one aims to use a neural network to approximate the score function. Naively, one can write down the following optimization objective, which is called the explicit score matching (ESM) formulation in [3]:

$$\mathbb{E}_{x\sim p(x)}\left[\frac{1}{2}\|s_\theta(x) - \nabla \log p(x)\|_2^2\right] \tag{1}$$

Explain why this particular formulation is often impossible to work with in many application scenarios of diffusion models.



C. (3pts) Prove that objective (1) is equivalent to the following objective, i.e. they differ only by a constant term that does not depend on the parameter θ:

$$\mathbb{E}_{x\sim p(x)}\left[\frac{1}{2}\|s_\theta(x)\|_2^2 + \mathrm{tr}(\nabla_x s_\theta(x))\right] \tag{2}$$

where $\nabla_x s_\theta(x)$ is the Jacobian of $s_\theta(x)$ with respect to its input x. This formulation is called the implicit score matching (ISM) in [3].

Hint: you need to use vector integration by parts and the fact that $p(x)$ vanishes at infinity.

c. We know that the explicit score matching formula: $J_{ESM} = E_{x\sim p(x)}\left[\frac{1}{2}\|s_\theta(x) - \nabla \log p(x)\|^2\right]$

If we define $s_\theta(x) = \nabla q(x)$, then we can write $J_{ESM} = E_{x\sim p(x)}\left[\frac{1}{2}\|\nabla \log p(x) - \nabla q(x)\|^2\right]$, where $q(x)$ is the model we are learning, and $p(x)$ is the true data distribution.

Expanding the squared term, we get: $J_{ESM} = E_{x\sim p(x)}\left[\frac{1}{2}(\|\nabla \log p(x)\|^2 + \|\nabla q(x)\|^2 - 2\nabla \log p(x)\nabla q(x))\right]$

Now, since $E_{x\sim p(x)}[\|\nabla \log p(x)\|^2]$ does not depend on the model $q(x)$, it can be ignored when minimising J_{ESM}.

So, when minimizing J_{ESM} we have:

$$J_{ESM} = E_{x\sim p(x)}\left[\frac{1}{2}\|\nabla q(x)\|^2 - \nabla \log p(x)\cdot \nabla q(x)\right] + \text{Constant} \quad -①$$

Next, to compute $E_{x\sim p(x)}[\nabla \log p(x)\cdot \nabla q(x)] = \int p(x)\,\nabla \log p(x)\,\nabla q(x)\,dx$ (By definition of expectation: $E_{p(x)}[g(x)] = \int g(x)\,p(x)\,dx$. In our case, $g(x) = \nabla \log p(x)\nabla q(x)$.)

Since we know $\nabla \log p(x) = \frac{\nabla p(x)}{p(x)}$ (By the logarithmic derivation formula which states that $\frac{d}{dx}\log f(x) = \frac{f'(x)}{f(x)}$) we have:

$$E_{x\sim p(x)}[\nabla \log p(x)\cdot \nabla q(x)] = \int p(x)\frac{\nabla p(x)}{p(x)}\nabla q(x)\,dx = \int \nabla p(x)\nabla q(x)\,dx$$

Now, applying integration by parts (or the divergence theorem), we get:

$\int \nabla p(x)\nabla q(x)\,dx = -\int p(x)\nabla\cdot(\nabla q(x))\,dx \rightarrow$ This is proven below *

Notice that by the definition of expectation (defined above) we have that $-\int p(x)\nabla\cdot(\nabla q(x))\,dx = -E_{x\sim p(x)}[\nabla\cdot\nabla q(x)]$

Therefore, we get: $E_{x\sim p(x)}[\nabla \log p(x)\cdot \nabla q(x)] = \int \nabla p(x)\nabla q(x)\,dx = -\int p(x)\nabla\cdot(\nabla q(x))\,dx = -E_{x\sim p(x)}[\underbrace{\nabla\cdot(\nabla q(x))}] = -E_{x\sim p(x)}[\Delta q(x)]$ where $\Delta q(x)$ is the laplacian of $q(x)$.

↳ This is the divergence of the gradient of $q(x)$ which is the laplacian of $q(x)$: $\Delta q(x)$. Explained below ■

Now, note that $\nabla q(x)$ is a vector of partial derivatives. The jacobian matrix $\nabla_x(\nabla q(x))$ is a matrix of second-order partial derivatives, which is the Hessian matrix of $q(x)$. The trace of the Jacobian w.r.t x ($tr\{\nabla_x \nabla q(x)\}$) is the sum of the diagonal elements of the Hessian matrix of $q(x)$.

We also know that the laplacian $\Delta q(x)$ is the sum of the diagonal elements of the Hessian matrix of $q(x)$

So, we can say that the trace of the Hessian is the Laplacian, i.e $tr\{\nabla_x \nabla q(x)\} = \Delta q(x)$.

So, we can say $E_{x\sim p(x)}[\nabla \log p(x)\cdot \nabla q(x)] = -E_{x\sim p(x)}[tr\{\nabla_x \nabla q(x)\}]$

Substituting this back into our Eqn for J_{ESM} (eqn ①), we get:

$$J_{ESM} = E_{x\sim p(x)}\left[\frac{1}{2}\|\nabla q(x)\|^2 - \nabla \log p(x)\cdot \nabla q(x)\right] = E_{x\sim p(x)}\left[\frac{1}{2}\|\nabla q(x)\|^2 + tr\{\nabla_x \nabla q(x)\}\right]$$

since we defined $s_\theta(x) = \nabla q(x)$, we can substitute this back in to get:

$$E_{x\sim p(x)}\left[\frac{1}{2}\|s_\theta(x)\|^2 + tr\{\nabla_x s_\theta(x)\}\right]$$

* For 2 scalar functions $u(x)$ & $v(x)$, the equivalent of integration by parts is the divergence theorem

The divergence theorem states $\int_V \nabla\cdot F\,dV = \int_{\partial V} F\cdot n\,ds$ where F is a vector field, V is a volume in space, ∂V is the boundary of V with outward pointing normal vector n.

We want to apply this to our integral: $\int \nabla p(x)\nabla q(x)\,dx$

Using the product rule for divergence: $\nabla\cdot(p(x)\nabla q(x)) = (\nabla p(x))\cdot(\nabla q(x)) + p(x)\nabla\cdot(\nabla q(x))$ [Product Rule for divergence states that $\nabla\cdot(p(x)F(x)) = (\nabla p(x))\cdot F(x) + p(x)\nabla\cdot F(x)$] (gradient of scalar func $p(x)$; $F(x)$: vector field; $\nabla\cdot F(x)$: divergence of the vector field)

Thus, $\int \nabla\cdot(p(x)\nabla q(x))\,dx = \int(\nabla p(x)\cdot\nabla q(x) + p(x)\nabla\cdot\nabla q(x))\,dx$

Since $p(x)$ is a PDF, it vanishes at infinity. So, the boundary term of the integral $\int_{\partial V} p(x)\nabla q(x)\cdot n\,dS$ goes to zero

So: $\int \nabla\cdot(p(x)\nabla q(x))\,dx = \int(\nabla p(x)\cdot\nabla q(x) + p(x)\nabla\cdot\nabla q(x))\,dx = 0$

Rearranging, we get:

$$\int \nabla p(x)\cdot\nabla q(x)\,dx + \int p(x)\nabla\cdot\nabla q(x)\,dx = 0 \Rightarrow \int \nabla p(x)\cdot\nabla q(x) = -\int p(x)\nabla\cdot\nabla f(x)\,dx$$

■ The laplacian (divergence of the gradient) of f can be defined by the sum of unmixed second partial derivatives:

(laplacian) $\Delta f = \nabla\cdot\nabla f = \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = tr(H(f))$ where H is the Hessian matrix. So applying this to our case, $\nabla\cdot\nabla q(x) = \nabla^2 q(x) = \Delta q(x) \Rightarrow$ laplacian of $q(x)$

D. (1pt) Explain why objective (2), despite being tractable to compute, may not be a desirable optimization objective to work with.

It is very hard to compute when the dimensions are high and the neural networks are deep. The ISM objective often requires integrating over high-dimensional spaces. Such integrals are typically intractable and cannot be solved analytically, necessitating the use of numerical approximation methods, which can be computationally expensive and challenging to implement effectively. In addition, the ISM objective involves expectations with respect to the data distribution, which can be hard to estimate unbiasedly, especially when dealing with high-dimensional data. The need for an unbiased estimator adds significant complexity to the learning process. So, it may also introduce a bias if there is high dimensionality of data involved.

E. (3pts) Now we focus on denoising score matching (DSM), which is an alternative way of score matching. In [3], the author established that the DSM formulation is equivalent to the ESM and ISM formulations. In this and the following subparts, you will see it from another slightly different angle based the following fact:

$$\mathbb{E}[x|\widetilde{x}] = \widetilde{x} + \sigma^2 \nabla \log p_\sigma(\widetilde{x})$$

where

$$\widetilde{x} = x + \sigma\varepsilon \quad \text{and} \quad \varepsilon \sim \mathcal{N}(0, I), \widetilde{x} \sim p_\sigma(\widetilde{x}).$$

This is known as the Tweedie's formula. Prove it.

Hint: You can use the following these steps:

1. Move $\widetilde{x}$ to the left hand side and use $\widetilde{x} = E[\widetilde{x}|\widetilde{x}]$ to simplify the left-hand side
2. Write the conditional expectation as an integral,
3. Apply Bayes' rule to show that the left hand side ($E[x|\widetilde{x}] - \widetilde{x}$, but simplified) can be written as $\int \frac{(x-\widetilde{x})p(\widetilde{x}|x)p(x)}{p(\widetilde{x})} dx$
4. Show that $\nabla \log p_\sigma(\widetilde{x}) = \int \frac{\nabla_{\widetilde{x}} p(\widetilde{x}|x)p(x)}{p(\widetilde{x})} dx$
5. You now should have very similar left and right hand sides. Expand what is needed and rearrange terms to get the desired result. Think about it: what is $\nabla_{\widetilde{x}} p(\widetilde{x}|x)$ in this case?

E. $E[x|\tilde{x}] = \tilde{x} + \sigma^2 \nabla \log p_\sigma(\tilde{x})$

$\Rightarrow E[x|\tilde{x}] - \tilde{x} = \sigma^2 \nabla \log p_\sigma(\tilde{x})$

$\Rightarrow E[x|\tilde{x}] - E[\tilde{x}|\tilde{x}] = \sigma^2 \nabla \log p_\sigma(\tilde{x})$

LHS: $E[x|\tilde{x}] - E[\tilde{x}|\tilde{x}] = E[x-\tilde{x}|\tilde{x}] = \int (x-\tilde{x})\, p(x|\tilde{x})\, dx$

Using bayes rule, we know that $p(x|\tilde{x}) = \frac{p(\tilde{x}|x) \cdot p(x)}{p(\tilde{x})}$. Substituting this into LHS:

$$\int (x-\tilde{x})\, p(x|\tilde{x})\, dx = \int (x-\tilde{x}) \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx$$

RHS: $\sigma^2 \nabla \log p_\sigma(\tilde{x}) = \sigma^2 \nabla \log \left(\int p(\tilde{x}|x)\, dx\right) = \sigma^2 \nabla \log \int p(\tilde{x}|x)\, p(x)\, dx = \sigma^2 \int \nabla_{\tilde{x}} \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx$

Note that since $p(\tilde{x}|x) \sim \mathcal{N}(x, \sigma^2)$, we can say $\nabla_{\tilde{x}} = \frac{x-\tilde{x}}{\sigma^2}$

LHS: $\int (x-\tilde{x}) \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx$ and RHS: $\sigma^2 \int \frac{x-\tilde{x}}{\sigma^2} \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx$

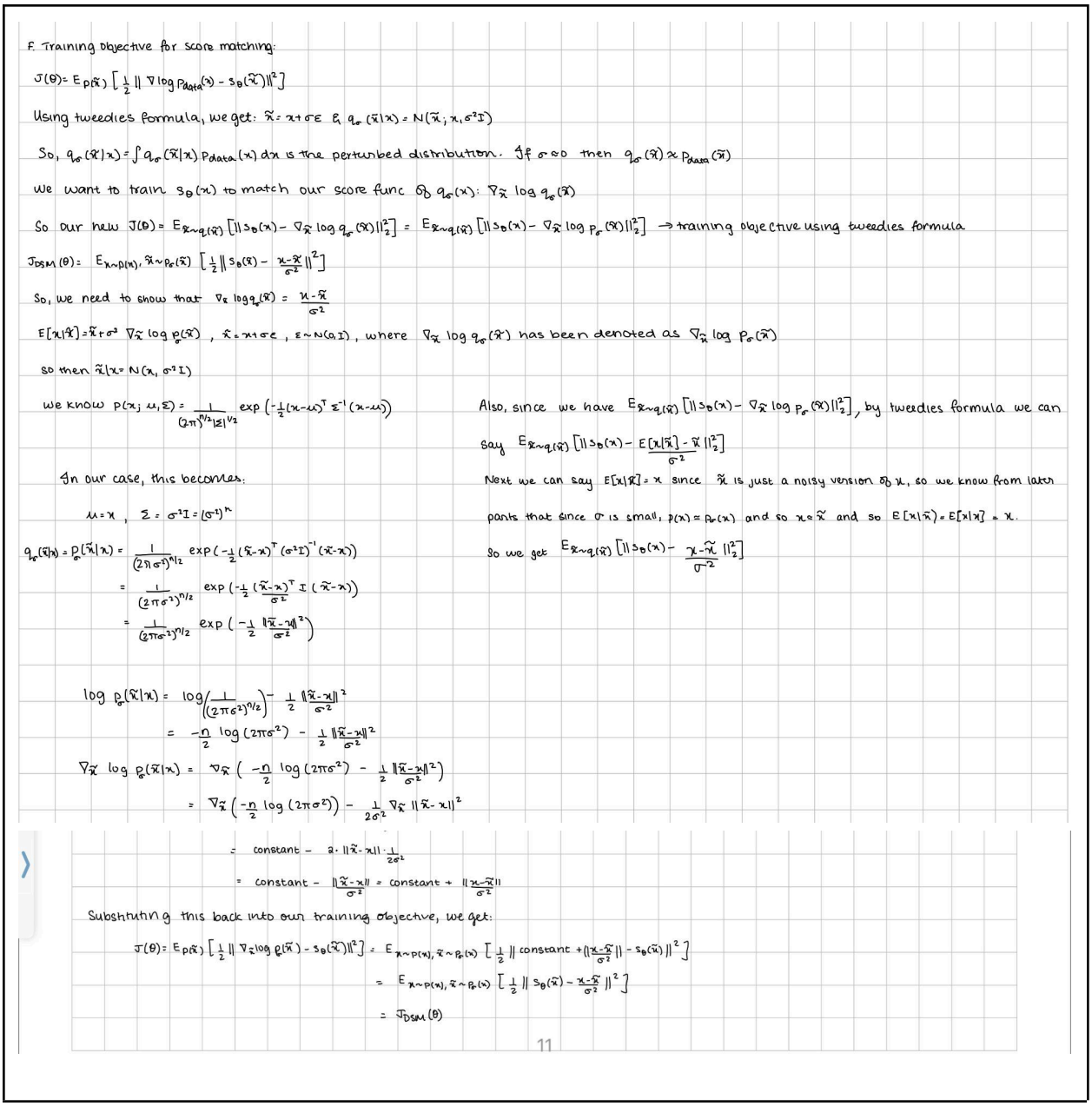
So, $\int (x-\tilde{x}) \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx = \int (x-\tilde{x}) \frac{p(\tilde{x}|x)\, p(x)}{p(\tilde{x})}\, dx$

and so we have proved tweedie's formula.

F. (3pts) Use Tweedie's formula to establish a training objective for score matching. Then prove that the established training objective is equivalent to the DSM objective:

$$\mathbb{E}_{x \sim p(x), \tilde{x} \sim p_\sigma(\tilde{x})} \left[\frac{1}{2} \left\| s_\theta(\tilde{x}) - \frac{x - \tilde{x}}{\sigma^2} \right\|_2^2 \right]$$

i.e. they differ only by a constant term that does not depend on the parameter θ.



G. (2pts) Compare the DSM objective with the simplified training objective at time t in DDPM with $x = \sqrt{\bar{\alpha}_t}x_0$, $\tilde{x} = x_t$, and $\sigma = \sqrt{1 - \bar{\alpha}_t}$. Show that the learned model $s_\theta(\tilde{x})$ in the DSM differs from the learned model $\varepsilon_\theta(x_t, t)$ by a constant factor involving σ.

More precisely, Show that there exists a scalar $c(\sigma) \in \mathbb{R}$, which depends on σ, such that if ε_{θ^*} is the optimizer for the DDPM objective at a certain time t, $s_{\theta^*} := c(\sigma)\varepsilon_{\theta^*}$ will be the optimizer for the DSM objective with $x = \sqrt{\bar{\alpha}_t}x_0$, $\tilde{x} = x_t$, and $\sigma = \sqrt{1 - \bar{\alpha}_t}$.

G. $x = \sqrt{\bar{\alpha}_t}\, x_0$, $\tilde{x} = x_t$ & $\sigma = \sqrt{1-\bar{\alpha}_t} \Rightarrow \sigma_t^2 = (1-\bar{\alpha}_t)$

In DDPM: $x = \sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t}\,\varepsilon$ where $\varepsilon \in N[0,I]$

In DSM: $\tilde{x} = x_t = \sqrt{\bar{\alpha}_t} x_{t-1} + \sigma\varepsilon = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$

The DDPM objective is: $L_t = E_{\varepsilon \in N[0,I]}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$

The DSM objective is: $J_{DSM} = E_{q(x_0, x_t|t)}\left[\|s_\theta(\tilde{x}) - \frac{x-\tilde{x}}{\sigma^2}\|_2^2\right]$

From the DDPM paper, we know $q(x_t|x_0)$, $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$ where $\varepsilon \sim N(0,I)$

$$x_t \Rightarrow \sqrt{1-(1-\bar{\alpha}_t)}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon \Rightarrow x_t = \sqrt{1-\sigma^2}\, x_0 + \sigma\varepsilon$$

So then the score func: $s_\theta(\tilde{x}) = \nabla_{\tilde{x}} \log q(x_t|x_0)$. For a gaussian distribution $N(x_t; \mu, \Sigma)$, the score func is $\nabla_{x_t} \log N(x_t; \mu, \Sigma) = -\Sigma^{-1}(x_t - \mu)$

For $q(x_t|x_0)$, we have $q(x_t|x_0) = N(x_t; \sqrt{1-\sigma^2}\, x_0, \sigma^2 I)$

So, score func $= \nabla_{\tilde{x}} \log q(x_t|x_0) = -(\sigma^2 I)^{-1}(x_t - \sqrt{1-\sigma^2} x_0) = \frac{-1}{\sigma^2}(x_t - \sqrt{1-\sigma^2} x_0)$

since $x_t = \sqrt{1-\sigma^2}\, x_0 + \sigma\varepsilon$, our score func: $\nabla_{\tilde{x}} \log q(x_t|x_0) = \frac{-1}{\sigma^2}(\sqrt{1-\sigma^2} x_0 + \sigma\varepsilon - \sqrt{1-\sigma^2} x_0) = \frac{-1}{\sigma^2}\sigma\varepsilon = -\frac{1}{\sigma}\varepsilon$

In DDPM, $\varepsilon_\theta(x_t, t)$ is the models estimate of the noise ε. So, we can write score func: $s_\theta(\tilde{x}) = \frac{-1}{\sigma}\varepsilon_\theta(x_t, t) \Rightarrow \varepsilon_\theta(x_t, t) = -\sigma\, s_\theta(\tilde{x})$. So, our $c(\sigma) = \frac{-1}{\sigma}$

So we have DDPM objective: $E_{t, x_0, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, t)\|^2\right] = E_{t, x_0, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$

DSM objective: $E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2}\|s_\theta(\tilde{x}) - \frac{1}{\sigma^2}(x-\tilde{x})\|_2^2\right]$

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2}\|s_\theta(\tilde{x}) - \frac{1}{1-\bar{\alpha}_t}(\sqrt{\bar{\alpha}_t} x_0 - x_t)\|_2^2\right]$

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon$

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2}\|s_\theta(\tilde{x}) - \frac{1}{1-\bar{\alpha}_t}(\sqrt{\bar{\alpha}_t} x_0 - \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\,\varepsilon)\|_2^2\right]$

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2}\|s_\theta(\tilde{x}) - \frac{1}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\|_2^2\right]$

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2}\|\frac{-1}{\sigma}\varepsilon_\theta(x_t, t) - \frac{1}{\sigma}\varepsilon\|_2^2\right]$

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2\sigma}\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right]$ — so, DSM objective similar to DDPM objective.

$= E_{x\sim p(x), \tilde{x}\sim p_\sigma(\tilde{x})}\left[\frac{1}{2\sigma}\|\varepsilon - (-\sigma s_\theta(\tilde{x}))\|_2^2\right]$

H. (3pts) Once you do the DSM for some noise level σ, you have an estimate of $p_\sigma(x)$. Let's see this enables us to do by looking a little into an (overdamped) Langevin dynamic:

$$dx_t = \nabla \log p_\sigma(x_t)dt + \sqrt{2}dB_t$$

where $\nabla \log p_\sigma(x)$ is the score function of the distribution $p_\sigma(x)$ and B_t is the n-dimensional Brownian motion. This is a stochastic differential equation (SDE) that admits the target distribution $p_\sigma(x)$ as its stationary distribution. Under some weak regularity conditions, the distribution of the solution to the SDE x_T at time T is $p_\sigma(x)$ as $T \to \infty$. When σ is small, $p_\sigma(x)$ is approximately the target distribution $p(x)$. Based on the above facts, describe an algorithm that approximately samples from the target distribution $p(x)$.

- Sample x_t (if no specific distribution is preferred, use standard Gaussian samples)
- For t = 1 to t = T:
  - Calculate $dx_t = \nabla \log p_\sigma(x_t)dt + \sqrt{2}dB_t$
  - x_{t+1} = x_t + dx_t

- Return x_T

More specifically,

1. Start with initial samples:
    a. Begin with an initial set of samples. If no starting distribution is provided, use standard gaussian samples.
2. Iterative updates:
    a. For t = 1 to t = T, perform the following updates:
        i. Calculate the score (gradient of log probability density) for the current samples
        ii. Adjust each sample by taking a step in the direction of the score, scaled by the step size
        iii. Add gaussian noise to each sample, scaled by the square root of twice the step size
        iv. Specifically, apply the langevin dynamics equation:
        $$dx_t = \nabla \log p_\sigma(x_t)dt + \sqrt{2}dB_t$$
3. The resulting samples, after sufficient iterations, will approximate the target distribution p(x) since when sigma is very small, p_sigma(x) is approximately p(x.

Part 2: Total 16/16 (36/85 + 15EC)

Part 3: The SDE perspective of diffusion models

(12pts)

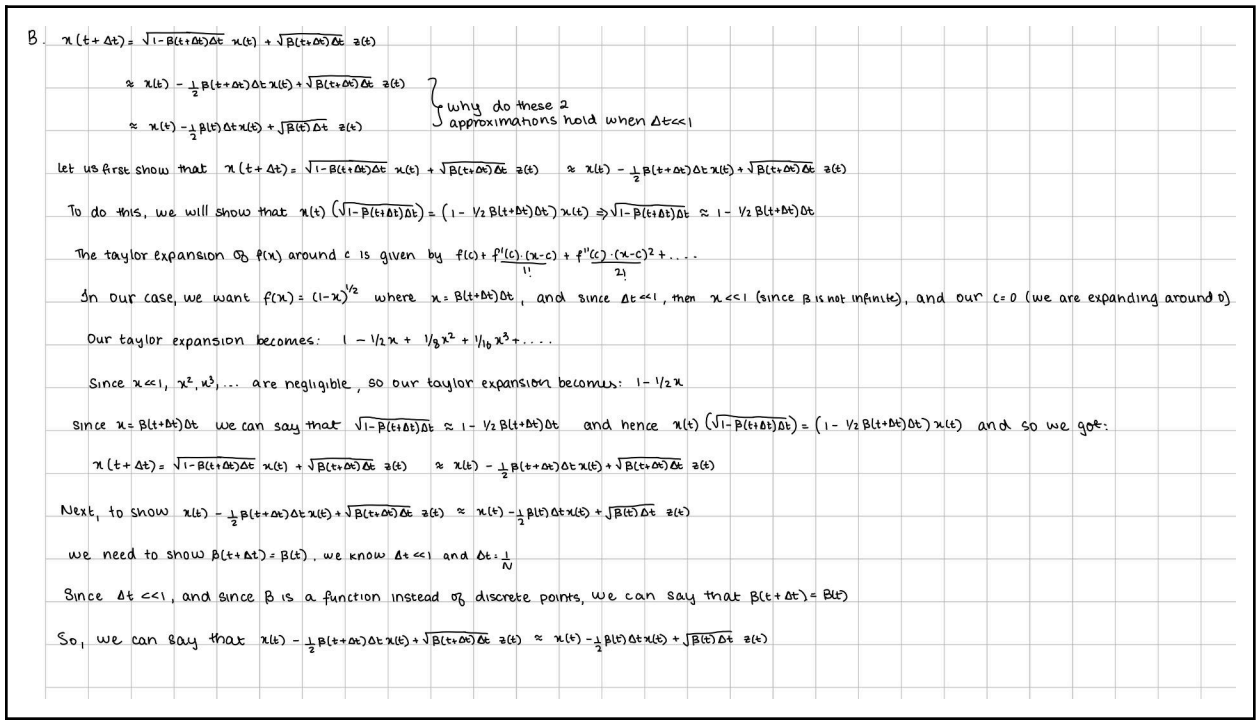
In Part 2, we have shown that the DSM is almost equivalent to the training of DDPM. In this part, you will see that, when the time discretization converges to 0, the forward process of DDPM is a diffusion SDE (subpart C). Furthermore, the backward (sampling) process of DDPM is equivalent to solving the so-called reverse SDE of the forward diffusion SDE that involves the score function (subpart D&E). So DDPM can be perfectly viewed from an SDE perspective and the equivalence between them is simply beautiful!

A. (0pt) Read through the paper [4]. Make sure you understand the main idea of the paper. In the following parts, you will prove some of the results in the paper.

B. (2pts) In Appendix B, the authors show that in the limit of time discretization Δt going to 0, the forward process of DDPM is equivalent to the variance-preserving (VP) SDE defined as

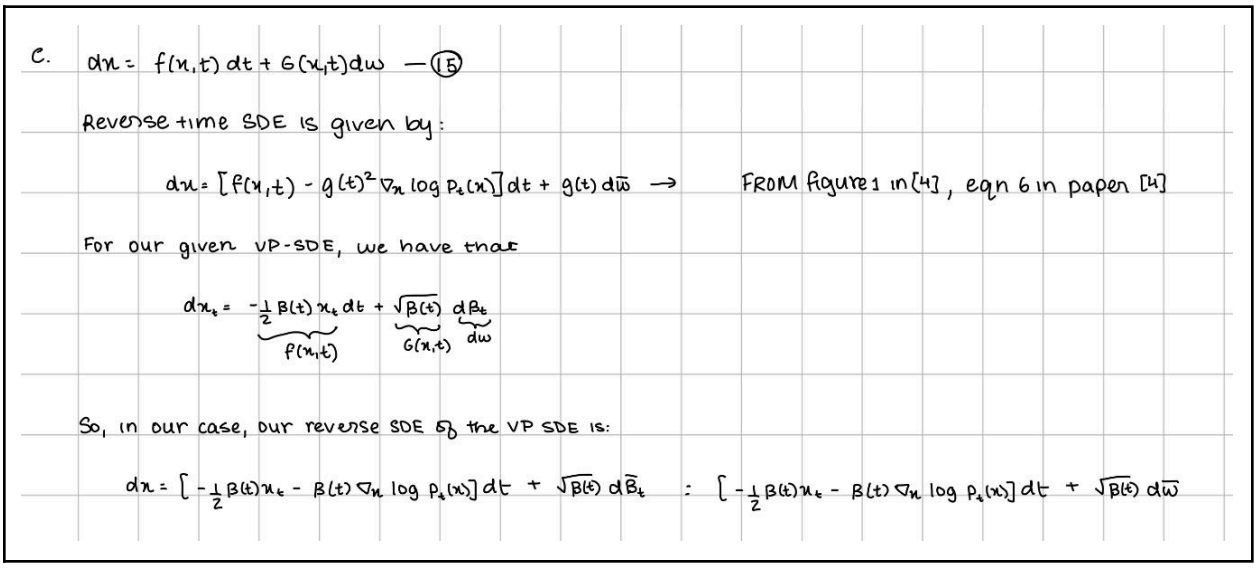
$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)}dB_t$$

with $\beta(t)$ defined in [4]. The authors showed this via two approximations (≈) and claimed that they hold when the step time step $\Delta t \ll 1$. Explain why those two approximations hold.



C. (1pt) A remarkable fact shown in [5] is that the reverse process of an diffusion process (SDE) is also an diffusion process (SDE). Here the reverse SDE means an SDE running backward in time from $t = T$ to $t = 0$ where T is the end time of the forward SDE. The distribution of the reverse SDE at each time step t is the same as that of the forward SDE

at time step t. The formula for the reverse SDE of a general SDE is shown in Equation (6) of [4]. Write down the reverse SDE of the VP-SDE accordingly.



D. (4pts) Fix the discretization Δt and let β_i be defined in the same way as in Appendix B&D of [4] ($\beta_i = \bar{\beta}_i \Delta t$, $x_i = x_t$, and $x_{i+1} = x_{t+\Delta t}$). Follow these steps to establish the equivalence between the reverse VP-SDE and DDPM sampling: (1) discretize the SDE you wrote down in the previous subpart, (2) substitute the true score function by the approximation s_θ, (3) arrange terms such that x_i is on one side and terms that involve s_θ,β_{i+1}, x_{i+1}, and z_{i+1} are on the other side, and (4) follow the derivation in Appendix E of [4] to conclude that your discretization of the VP-SDE is equal to the following update rule when $\beta_i \to 0$, i.e. $\Delta t \to 0$:

$$\mathbf{x}_i = \frac{1}{\sqrt{1-\beta_{i+1}}}(\mathbf{x}_{i+1} + \beta_{i+1}\mathbf{s}_{\boldsymbol{\theta}*}(\mathbf{x}_{i+1}, i+1)) + \sqrt{\beta_{i+1}}\mathbf{z}_{i+1}$$

Note: the above equation is a screenshot from [4]. Here we do not distinguish θ^* and θ.

D. $dx = \left[-\frac{1}{2}\beta(t)x_t - \beta(t)\nabla_x \log p_t(x)\right]dt + \sqrt{\beta(t)}\,dw$

$= \left[-\frac{1}{2}\bar{\beta}_{Nt}\Delta t\, x_i - \bar{\beta}_{Nt}\Delta t\, \nabla_x \log p_t(x)\right]\Delta t + \sqrt{\bar{\beta}_{Nt}\Delta t}\,(z_{i+1})$

$= \left[-\frac{1}{2}\beta_i x_i - \beta_i \nabla_x \log p_t(x)\right]\Delta t + \sqrt{\beta_i}\,(z_{i+1})$

$x_{i+1} - x_i = \left[-\frac{1}{2}\beta_i x_i - \beta_i s_\theta(x)\right]\Delta t + \sqrt{\beta_i}\,(z_{i+1})$

$x_{i+1} - x_i = -\frac{1}{2}\beta_i x_i \Delta t - \beta_i s_\theta(x)\Delta t + \sqrt{\beta_i}\,(z_{i+1})$

$x_{i+1} = x_i + \left[-\frac{1}{2}\beta_i x_i - \beta_i s_\theta(x_i)\right]\Delta t + \sqrt{\beta_i}\,(z_{i+1})$

Using eqn 46 in appendix E, we get:

$x_i = x_{i+1} + \frac{1}{2}\beta_{i+1}x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i+1) + \sqrt{\beta_{i+1}}\,(z_{i+1})$

$\Rightarrow x_i = \left(1+\frac{1}{2}\beta_{i+1}\right)(x_{i+1}) + \beta_{i+1}s_\theta(x_{i+1}, i+1) + \sqrt{\beta_{i+1}}\,(z_{i+1})$

$\approx x_i = \left(1+\frac{1}{2}\beta_{i+1}\right)(x_{i+1}) + \beta_{i+1}s_\theta(x_{i+1}, i+1) + \frac{1}{2}\beta_{i+1}^2 s_\theta(x_{i+1}, i+1) + \sqrt{\beta_{i+1}}\,(z_{i+1})$ — we can add the squared term since β_{i+1} is really small.

$\Rightarrow x_i = \left(1+\frac{1}{2}\beta_{i+1}\right)\left(x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\,(z_{i+1})$

$\approx x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\,(z_{i+1})$ (we get $1+\frac{1}{2}\beta_{i+1} = \frac{1}{\sqrt{1-\beta_{i+1}}}$ from the taylor expansion of $\frac{1}{\sqrt{1-\beta_{i+1}}}$ about 0 *)

* let $\beta_{i+1} = a$, and we know $a \ll 0$ as $\beta_t \to 0$ & $\Delta t \to 0$

Taylor expansion of $\frac{1}{\sqrt{1-a}}$ around $0 = 1 + \frac{1}{2}a + \frac{3a^2}{8} + \frac{5a^3}{16} + \ldots$

Since $a \ll 1$, $a^2, a^3, a^4 \ldots$ terms can be ignored so we are left with $1+\frac{1}{2}a = 1+\frac{1}{2}\beta_{i+1}$

Thus we have shown $x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\,(z_{i+1})$

E. (1pt) This update rule is almost the same as that in the sampling algorithm (Algorithm 2) in [1] when the σ_t variable in [1] is chosen to be $\sigma_t = \sqrt{\beta_t}$. The only difference is the coefficient in front of s_θ. Use your answer in Part 2G to justify this difference. After proving this, you should be able to see the connection between the SDE formulation and DDPM!

Hint: Think about what σ is in the context of DDPM. Note that σ is the standard deviation of the noise added to the clean image (denoted as x_0 in DDPM).

E. In our SDE formulation, the term corresponding to the score func is $\beta_{i+1} s_\theta(x_{i+1}, i+1)$

In DSM, our score func. was $\frac{-1}{\sigma}\epsilon_\theta(x_t, t) = s_\theta(x_t)$

we know $\sigma_t = \sqrt{\beta_t}$ in DDPM and that $\sigma = \sqrt{1-\bar{\alpha}_t}$ (from 2G)

$$x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \beta_{i+1} s_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\,(z_{i+1})$$

Let $s_\theta(x) = \frac{-1}{\sqrt{\beta_{i+1}}}\epsilon_\theta(x_t, t)$, since $\sigma_t^2 = \beta_t \Rightarrow \frac{1}{\sigma_{t+1}} = \frac{1}{\sqrt{\beta_{t+1}}} \Rightarrow s_\theta(x) = \frac{-1}{\sigma}\epsilon_\theta(x_t, t) = \frac{-1}{\sqrt{\beta_{i+1}}}\epsilon_\theta(x_t, t)$

we know from above that SDE update rule is: $x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \beta_{i+1} s_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\, z_{i+1}$

substituting $s_\theta(x) = \frac{1}{\sqrt{\beta_{i+1}}}\epsilon_\theta(x_t, t)$,

we get: $x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \beta_{i+1}\frac{1}{\sqrt{\beta_{i+1}}}\epsilon_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\, z_{i+1}$

$$x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \sqrt{\beta_{i+1}}\,\epsilon_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\, z_{i+1}$$

$$x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}\left(x_{i+1} + \frac{\beta_{i+1}}{\sqrt{\beta_{i+1}}}\epsilon_\theta(x_{i+1}, i+1)\right) + \sqrt{\beta_{i+1}}\, z_{i+1}$$

we have that $\epsilon_\theta(x_t, t) = -\sigma s_\theta(x_t)$ and $\sigma_t^2 = \beta_t \Rightarrow \sigma_{i+1} = \sqrt{\beta_{i+1}}$

$$x_t = \frac{1}{\sqrt{1-\beta_{t+1}}}\left(x_{t+1} - \frac{\beta_{t+1}}{\sigma} \cdot -\sigma s_\theta(x_t)\right) + \sqrt{\beta_{t+1}}\, z_{t+1}$$

$$x_t = \frac{1}{\sqrt{1-\beta_{t+1}}}\left(x_{t+1} + \beta_{t+1} s_\theta(x_t)\right) + \sqrt{\beta_{t+1}}\, z_{t+1}$$

F. (2pts) Langevin dynamics was already used before diffusion models as a way to sample from a distribution by learning only the score function $\nabla \log p(x)$ of a distribution instead of explicitly modeling the distribution by learning the log probabilities $\log p(x)$.

Diffusion generates better samples because it leverages more information. To see this, compare the sampling procedure in the reverse SDE with the Langevin diffusion in Part 2H. Briefly explain why the reverse SDE approach leverages more information than the Langevin dynamics for sampling a distribution.

As you have already established in 3E, there is a connection between the SDE formulation and DDPM, as they're both using diffusion. Therefore, the extra information that diffusion leverages when compared to Langevin dynamics is reflected in both the score matching objective in Equation (7) of [4] and the training objective of DDPM in Equation (14) of [1]. Explain how this is reflected.

Langevin Dynamics

Langevin dynamics is a method to sample from a distribution $p(x)$ by iteratively updating samples using the gradient of the log-probability (the score function). The update rule for Langevin dynamics is:

$$x_{t+1} = x_t + \frac{\epsilon}{2}\nabla \log p(x_t) + \sqrt{\epsilon}\eta_t$$

where η_t is a Gaussian noise term.

Reverse SDE Approach

The reverse SDE approach, used in diffusion models, is designed to model the process of generating data by reversing a diffusion process. The forward process corrupts data by gradually adding noise, modeled by an SDE, and the reverse process aims to denoise it.

The reverse SDE can be expressed as:

$$dx = [f(x,t) - g^2(t)\nabla \log p_t(x)]dt + g(t)d\tilde{B}_t$$

where $f(x,t)$ and $g(t)$ are drift and diffusion coefficients, and $\nabla \log p_t(x)$ is the time-dependent score function of the noisy distribution $p_t(x)$.

Why Reverse SDE Leverages More Information

Time Dependency

In the reverse SDE, the score function $\nabla \log p_t(x)$ is explicitly time-dependent, capturing how the distribution evolves over time. Langevin dynamics, in contrast, typically uses a static score function $\nabla \log p(x)$.

Drift and Diffusion Terms

The reverse SDE includes both a drift term $f(x,t)$ and a diffusion term $g(t)$, which are functions of time. These terms allow the model to more accurately reverse the noise addition process, leveraging more detailed information about how the data was corrupted.

Score Matching Objective

The score matching objective in diffusion models incorporates information from multiple noise levels, as shown in Equation (7) of Vincent et al. (2011):

$$\mathcal{L}_{SM} = E_{q(\tilde{x}_t|x)}\left[\frac{1}{2\sigma_t^2}\|s_\theta(\tilde{x}_t,t) - \nabla \log q(\tilde{x}_t|x)\|^2\right]$$

This objective trains a score model s_θ to predict the score function across different noise levels, providing a richer training signal compared to the static score function used in Langevin dynamics.

Training Objective of DDPM

The training objective of Denoising Diffusion Probabilistic Models (DDPM), as given in Equation (14) of Ho et al. (2020):

$$\mathcal{L}_{DDPM} = E_{t,x_0,\epsilon}\left[\left\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\right\|^2\right]$$

This objective is a reweighted variational lower bound that incorporates the time-dependent noise schedule $\bar{\alpha}_t$, which changes the distribution at each timestep t. The model learns to denoise progressively, starting from heavily corrupted data.

So basically,

The reverse SDE approach leverages more information than Langevin dynamics because it uses a time-dependent score function, includes explicit drift and diffusion terms, and has richer training objectives that account for noise levels across different time steps. These aspects allow diffusion models to generate higher-quality samples by effectively reversing the corruption process, capturing the evolution of the data distribution over time.

G. (2pts) With the SDE perspective, we can better understand how conditional diffusion models work. Specifically, consider the following conditional reverse diffusion process on some conditional signal y:

$$dx_t = -\left[f(x_t, t) - g(t)^2 \nabla_{x_t} \log p_t(x_t|y)\right] dt + g(t) d\bar{B}_t$$

where $d\bar{B}_t$ is the time-reversed Brownian motion. The solution to this reversed SDE is $p(x|y)$ the desired conditional distribution on y. Using Bayes' rule to briefly explain the intuition of how the classifier guidance diffusion mentioned in Lecture 12 works.

Bayes' rule states:

$$p(x|y) = \frac{p(y|x) \cdot p(x)}{p(y)}$$

In our case, we're interested in the conditional distribution $p_{\theta,\phi}(x_{t-1}|x_t, y)$, which represents the probability density of x_{t-1} given x_t and y. To obtain this conditional distribution, we can apply Bayes' rule:

$$p_{\theta,\phi}(x_{t-1}|x_t, y) = \frac{p_\theta(x_{t-1}|x_t) \cdot p_\phi(y|x_t)}{p_\phi(y)}$$

By defining the normalization constant Z as:

$$Z = \frac{1}{p_\phi(y)}$$

we can express the conditional distribution as:

$$p_{\theta,\phi}(x_{t-1}|x_t, y) = Z \cdot p_\theta(x_{t-1}|x_t) \cdot p_\phi(y|x_t)$$

1. The model is trained on a dataset by adding noise to the samples in a forward process, turning them into a noise distribution (typically Gaussian).

2. The reverse process, learned through denoising, trains a neural network to predict how to reverse the noise, effectively learning the score function $\nabla_{x_t} \log p(x_t)$, which represents the gradient of the log-prior.

Classifier guidance incorporates the pretrained classifier $p_\phi(y|x_t)$ to influence the generative process towards producing samples more likely to match the condition y.

The score function for the conditional distribution $p_\theta(x_{t-1}|x_t)$ is obtained by combining the gradients of the log-prior and the log-likelihood:

$$\nabla_{x_t} \log p_{\theta,\phi}(x_{t-1}|x_t, y) = \nabla_{x_t} \log p_\theta(x_{t-1}|x_t) + \nabla_{x_t} \log p_\phi(y|x_t)$$

- $\nabla_{x_t} \log p_\theta(x_{t-1}|x_t)$: The score function for the prior $p(x_{t-1}|x_t)$, learned during the training of the diffusion model.

- $\nabla_{x_t} \log p(y|x_t)$: The gradient of the log-likelihood from the classifier.

During the reverse diffusion process, we use the combined score function to adjust the reverse SDE:

$$s(x_t, y, t) = \nabla_{x_t} \log p_\theta(x_{t-1}|x_t) + \nabla_{x_t} \log p_\phi(y|x_t)$$

This score guides the sampling from the reverse SDE:

$$dx_t = \left(f(x_t, t) + g(t)^2 s(x_t, y, t)\right)\, dt + g(t)\, d\overline{B}_t$$

So essentially,

- **Prior** $p_\theta(x_{t-1}|x_t)$: Learned during the training of the diffusion model and represented by the score function $\nabla_{x_t} \log p_\theta(x_{t-1}|x_t)$.
- **Likelihood** $p_\phi(y|x_t)$: Provided by a pretrained classifier, used to adjust the generative process through its gradient.
- **Combining Terms**: The conditional score function combines the prior and likelihood gradients to form $\nabla_{x_t} \log p(x_t|y)$.
- **Guided Sampling**: The modified score function is used in the reverse SDE to generate samples conditioned on y, effectively steering the generative process using Bayes' rule.

This combined approach allows the diffusion model to produce samples that match the desired condition y by integrating the prior distribution and classifier guidance into the reverse diffusion process.

Note: if one uses a function $s_\theta(x_t, y, t)$ to approximate $\nabla_{x_t} \log p_t(x_t|y)$ via score matching and then sample the reverse SDE with $s_\theta(x_t, y, t)$, it is called the classifier-free sampling, which is also mentioned in Lecture 12.

H. (0pt) A lot of the recent works on diffusion models also builds on the probability flow (PF) ODE formulation in [4]. We don't cover it this assignment, but read Section 4.3 and Appendix D of [4] if you want to learn more about diffusion models!

Part 3: Total 12/12 (48/85 + 15EC)

Part 4: Code your own diffusion models (total: 36pts, EC:10pts)

In this part, you will implement a Score-Matching diffusion model, as proposed by Song et al in 2021 [4], to generate new greyscale images. We will be using the Variance Preserving (VP) stochastic differential equation that you studied in Part 3 to model the diffusion.

Colab link

As in previous assignments, you should develop locally on an IDE, then run training on Google Colab Pro+. Submitted code that does not run without error will not be considered valid for grading.

A. Defining the VP SDE (total: 8 pts)

Make sure you read Song21 [4] and understand how the Variance Preserving (VP) stochastic differential equation is defined. Pay attention to the drift and diffusion coefficients.

Variance Preserving diffusion SDE (Eq 11 in [Song21]):

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}d\mathbf{w}$$

This follows the general SDE form $d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)d\mathbf{w}$. We call $f(\mathbf{x}, t)$ the **drift coefficient** and $g(t)$ the **diffusion coefficient**.

The corresponding general conditional linear Gaussian distribution is:

$$p(x_t|x_0) = \mathcal{N}(x_t; \alpha(t)x_0, \sigma^2(t)I)$$

where $\alpha : [0, 1] \to \mathbb{R}, \sigma : [0, 1] \to \mathbb{R}$

$\mu(t), \sigma(t)$ can be derived analytically from $f(\mathbf{x}, t), g(t)$.

$$\begin{cases} \mu(t) = \alpha(t)x_0 = \exp(c(t))x_0 \\ \sigma^2(t) = 1 - \exp(2c(t)) \end{cases}$$

i. In the case of VP, specify the drift and diffusion coefficient. (1 pts)

Drift coefficient: -½ β(t)x diffusion coefficient: $\sqrt{\beta}$(t)

ii. In the case of VP, what are β(t) and c(t)?: (2 pts)
(Hint: Refer to Equations (32) and (33) in Song21)

β(t) = β_min + t(β_max - β_min), $c(t) = \frac{-1}{4}t^2(B_{max} - B_{min}) - \frac{1}{2}tB_{min}$, where B_max and B_min are B{bar}_max and B{bar}_min, for t in [0,1]

iii. Fill in the TODOs in the VP class. (5 pts)

Done in code submission

B. Defining the VP sampling (total: 16 pts)

Make sure you read Appendix E of Song21 [4] and understand how to generate samples. You will implement two sampling methods in this part of the assignment: Euler-Maruyama (EM) and Predictor-Corrector (PC) with EM as the predictor.

i. EM sampler:
Go over the iteration rule.

As per Appendix E of [Song21], recall that for any SDE of the form

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w},$$

the reverse-time SDE is given by

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}}.$$

We use the Euler-Maruyama numerical method to solve for the reverse-time SDE. This method relies on discretizing the SDE, replacing dt with Δt and $d\mathbf{w}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, g^2(t)\Delta t\mathbf{I})$.

This lead to the following iteration rule:

$$\mathbf{x}_{t-\Delta t} = \mathbf{x}_t - \mathbf{f}(\mathbf{x}_t, t)\Delta t + g^2(t)s_\theta(\mathbf{x}_t, t)\Delta t + g(t)\sqrt{\Delta_t}\mathbf{z}_t.$$

Note: for the last step (i.e. $t - \Delta_t = 0$), we do not wish to add back noise ($g(t)\sqrt{\Delta_t}\mathbf{z}_t$).

Fill in the TODOs in the Euler_Maruyama_sampler method. (8 pts)

Notice that the analytical solutions for $\sigma(t)$ depends on $c(t)$. As a result, $x(t = 1) = z \cdot \sigma(t = 1)$ with $z \sim N(0, 1)$.

ii. PC sampler:
Go over the PC algorithm (Algorithm 5 in Song21).
Fill in the TODOs in the predictor_corrector_sampler method. (8 pts)

C. Generating samples (total: 12 pts)

We will be training our model for 50 epochs, LR=1e-4 (these have been filled in for you in the notebook) with the VP SDE on the FashionMNIST dataset. Feel free to experiment with early stopping + patience. Remember to save your checkpoints.
Two suggestions for [beta_min, beta_max]: [0.01, 5], [0.01, 10]. Again up to you to experiment.
*Note: training a score model for 50 epochs on FashionMNIST takes **~10 min** on an A100 GPU.*

i. We are working with FashionMNIST. Plot 64 images from the dataset to get an idea of the prior distribution. What are the different classes? What are the dimensions of each image? Include your plot in the submission PDF. (3 pts)

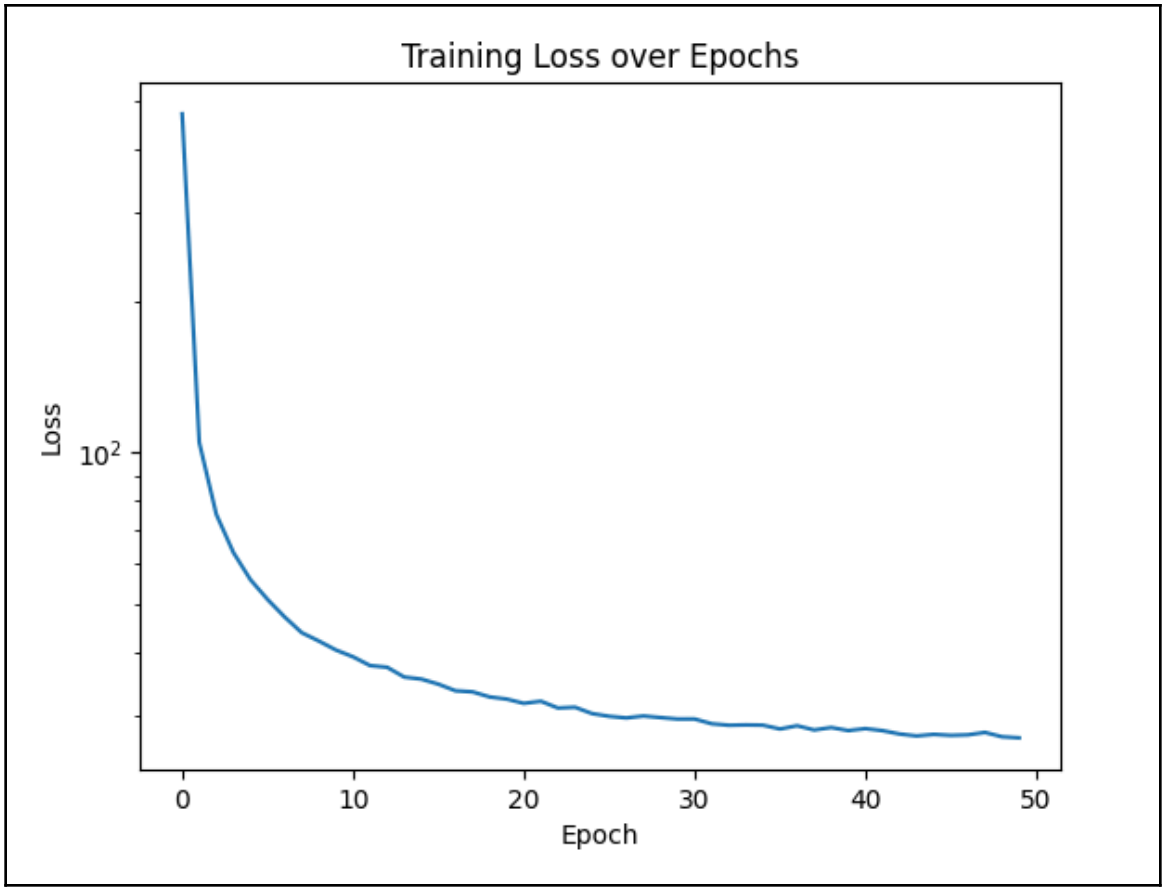


Dimensions: 28x28 grayscale images
Dataset has 60,000 images and 10 classes:

Label	Description
0	T-shirt/top
1	Trouser
2	Pullover

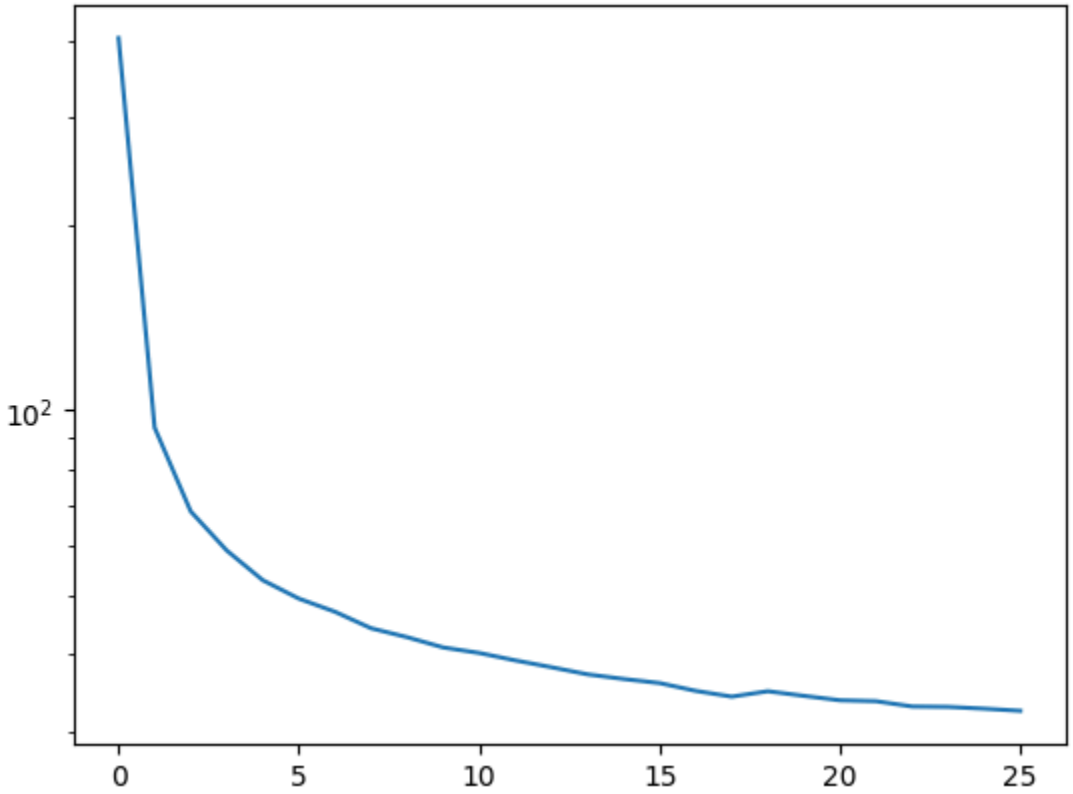
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

ii. Plot your losses (log scale) over epochs. Discuss any early stopping and patience strategy you implemented. Specify your [beta_min, beta_max] and any hyperparameter you modified. Include your titled plot in the submission PDF. (3 pts)



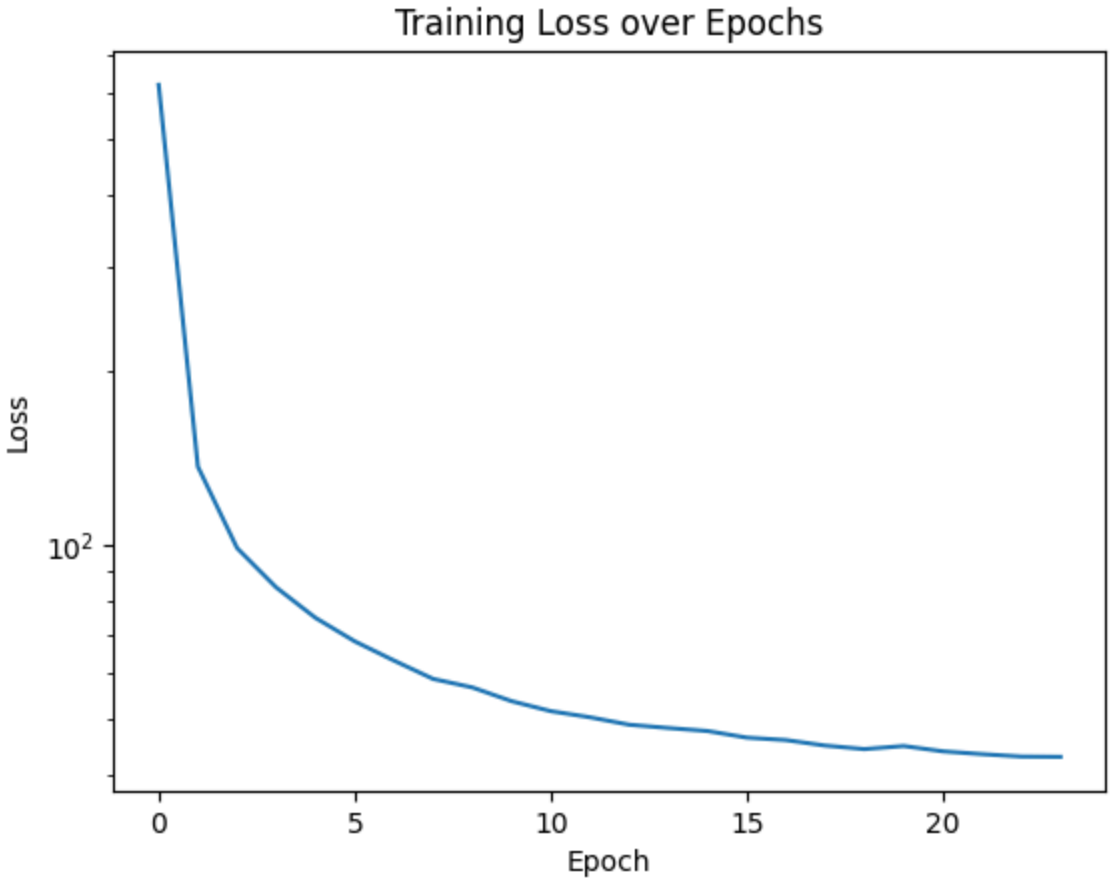
```
beta_min = 0.1
beta_max = 20
```

My early stopping patience is 10 for the above.




```
beta_min = 0.1
beta_max = 20
```

My early stopping patience is 3 for the above.

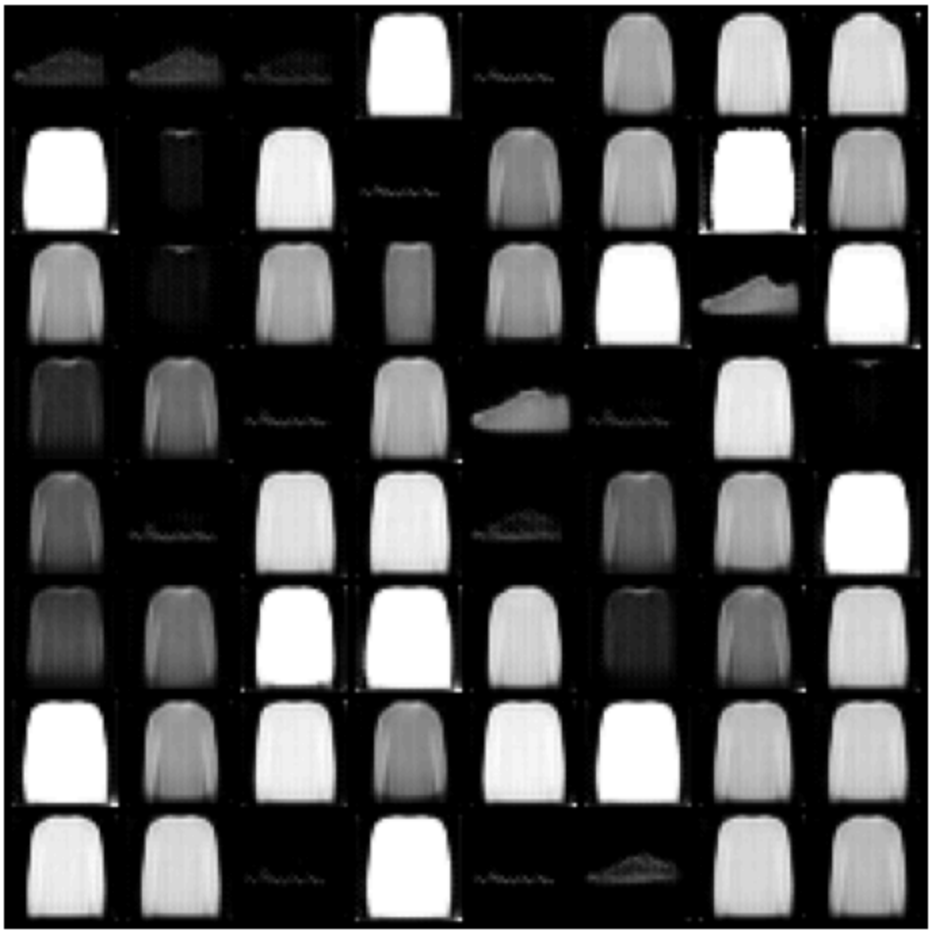



```
beta_min = 0.01
beta_max = 10
```

My early stopping patience is 3 for the above.

I chose to use an early stopping of 3 since it improved my performance. At 10, I felt like my model was overfitting. In addition, the loss does not reduce as much after 25 epochs so if we keep going there is a higher chance of overfitting.

iii. Generate 64 samples with your EM sampler. Include your plot in the submission PDF. (1 pt)



iv. Generate 64 samples with your PC sampler for at least 2 different numbers of corrector steps. Include your plots in the submission PDF. (2 pts)

```
Num_corrector_steps = 1
```



Num_corrector_steps = 1



Num_corrector_steps = 10



Num_corrector_steps = 5



Num_corrector_steps = 5



v. Discuss any qualitative difference you observe in your samples from using different samplers, and from varying the number of corrector steps. Referring to your plot in (i), comment on what your model captures well from the prior distribution and what it struggles with. (3 pt)

1. Euler-Maruyama Sampler:
    - Trajectory: For high-dimensional data like FashionMNIST, the EM sampler produces noisier and less structured images. The lack of a corrective mechanism means each step might introduce accumulated errors, resulting in blurry or unrealistic samples, with just the silhouette showing.
    - Sample Quality: Samples generated may fail to capture fine-grained details and complex structures inherent in fashion images, leading to outputs that lack clear boundaries or distinct features (e.g., edges of clothing items).
2. Predictor-Corrector Sampler:
    - Trajectory: The PC sampler, with its alternating prediction and correction steps, produces more coherent and realistic images. Each correction step helps align the samples closer to the true data distribution, refining the details and reducing noise.
    - Sample Quality: Improved with more corrector steps, resulting in samples that better capture textures, edges, and intricate details of the FashionMNIST items. The images are likely to appear sharper and more accurate representations of the clothing items.

1. Few Corrector Steps:
    - Sample Appearance: Images still contain significant noise, with blurred boundaries and less defined features. Some structural aspects of clothing items might be recognizable, but the overall quality is worse with fewer steps.
    - Comparison to EM: Although there is an improvement over the EM sampler, the difference is not as substantial with fewer corrector steps.
2. Many Corrector Steps:
    - Sample Appearance: Increased corrector steps lead to images that are more refined, with sharper edges and clearer features such as patterns, textures, and shapes of clothing items. The quality of the samples improves significantly, providing a more accurate depiction of the dataset.
    - Diminishing Returns: Beyond a certain point, the improvement in sample quality might become marginal relative to the computational cost, but generally, more corrector steps ensure better adherence to the data distribution.

Capturing the Prior Distribution:

- What the Model Captures Well:
    - Overall Structure: The model tends to capture the general structure and silhouette of the clothing items. Basic shapes and categories (e.g., T-shirts, basic shoes) are recognizable.
    - Primary Modes: The most prominent features and modes of the data distribution are captured, leading to a general resemblance to the original dataset.
- Struggles:
    - Fine Details: The model struggles with fine-grained details such as textures, intricate patterns, and small features, especially with fewer corrector steps.
    - Complex Dependencies: Capturing the complex dependencies and variations in the dataset (like variations in fabric patterns or subtle differences between similar items) is hard.
    - Edges and Boundaries: Without sufficient correction, edges and boundaries of clothing items appear blurred or less defined.

D: Diffusion models for inverse problem (EC: 10pts)

<u>Important</u>: Do NOT attempt until you are <u>entirely</u> satisfied with your work for <u>all</u> of the rest of the assignment (Parts 1 through 5).

<u>Important</u>: Minimum guidance will be provided in OH or on Piazza.
🎶🎵You're on your own, kid. You always have been.🎵🎶

This part will require you to read and understand [Song22] Solving Inverse Problems In Medical Imaging With Score-Based Generative Models. Warning: This is not an easy paper, and it contains medical jargon.

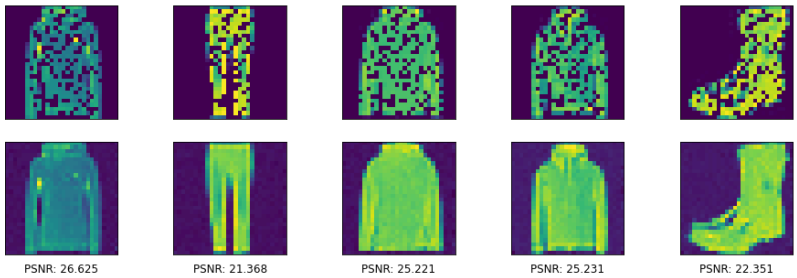


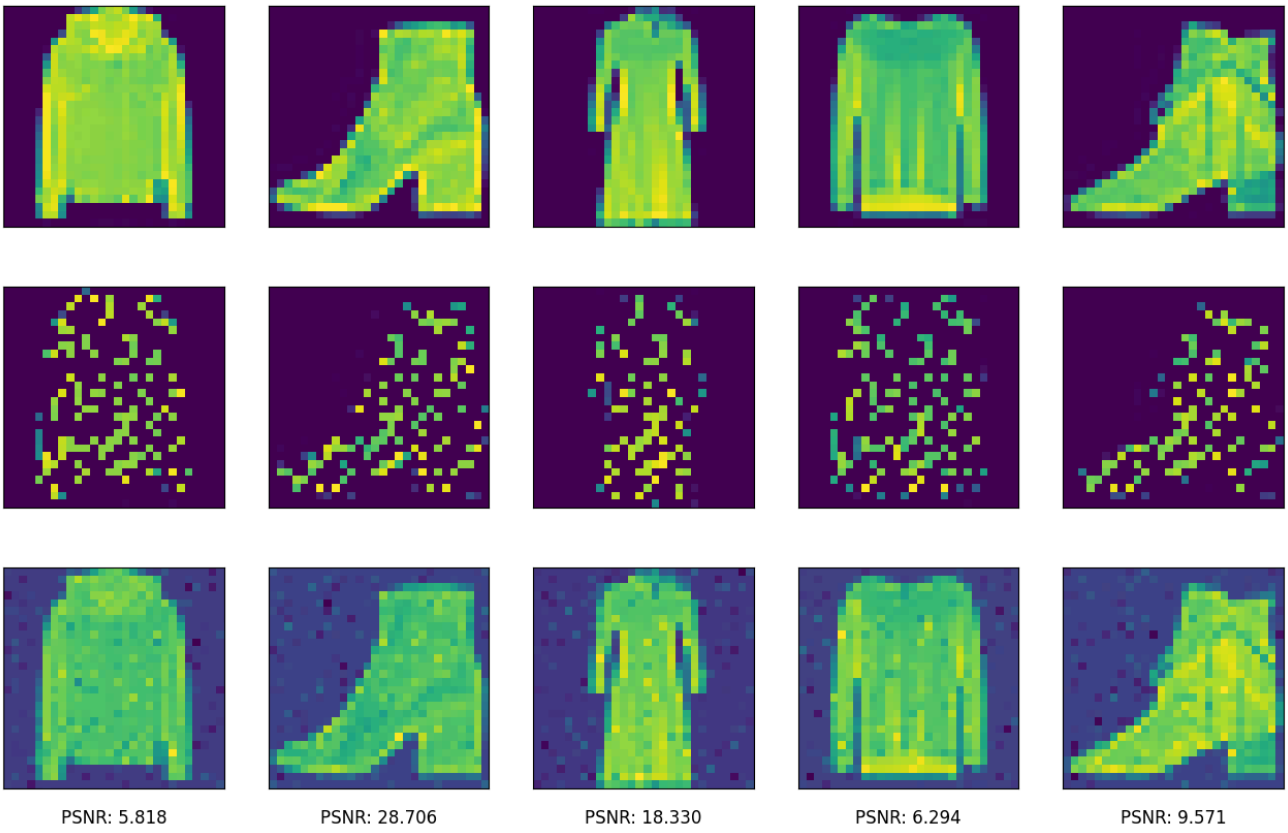
Figure: Example of (top) corrupted images with a matrix A inpainting 25% of pixels, and (bottom) recovered images with a conditional score-matching diffusion model

In our toy example, our measurement matrix A will be an explicit inpainting matrix that replaces some of the pixels in an image by 0 (blackening them out). We will set the corruption rate at 75%!

Your job will be to modify the sampling process to include conditioning on the perturbed measurements at various t, so that the reverse diffusion process generates recovered images when conditioned on corrupted images.

<u>Expectations</u>: The expected result is a plot with 3 rows: clean images, corrupted images, and reconstructed images. The plot must include the PSNR of each reconstructed image. You should use at least 5 clean images. We wrote the plotting function for you. We also expect you to explain and justify your implementation choices. Everything (plot, explanations, justifications) must appear in your submission PDF.

See the Colab notebook for more info.



PSNR Calculations:
Explanation: The PSNR (Peak Signal-to-Noise Ratio) is used to measure the quality of the reconstructed images. A higher PSNR indicates a better reconstruction. The function computes the mean squared error (MSE) between the clean and noisy images, and then calculates the PSNR using the logarithmic ratio. Justification: PSNR is a standard metric for image quality assessment, providing an objective measure of the reconstruction quality. The use of a small epsilon value helps avoid division by zero.

Setting Up Time Step and Batch Size:
Explanation: The time step and batch size are set up for the diffusion process. Here, time_step is set to 1, and batch_size is 1, indicating that the process will be applied to one image at a time. Justification: A single time step and batch size make the process easier to debug and understand. This incremental approach ensures that each image is processed individually, facilitating more detailed observation of the inpainting and reconstruction.

Diffusion Process Setup:
Explanation: The sde (Stochastic Differential Equation) is initialized with parameters suitable for variance-preserving diffusion.

Justification: The specific parameters for the VP SDE (0.1, 10, 1000) are chosen based on standard practices in score-based generative modeling. These parameters ensure that the diffusion process operates within a stable range, preserving the variance of the data. Also, my model in previous parts performed well with 0.1, 20 and 0.01, 10 so I chose to try 0.1, 10 for this.

Image Reconstruction:
Explanation: Each corrupted image is processed individually through the condition_on_inpainted_y function, which conditions the reverse diffusion on the inpainted measurements. The reconstructed images are collected into a list and then stacked into a tensor.
Justification: Processing each image individually ensures that the conditioning is accurately applied based on the specific corrupted input. This approach aligns with the principles of conditional score-based generative models, where the conditioning helps guide the reverse diffusion process towards reconstructing the original image.

Part 5: Diffusion models on a larger scale (total: 12pts)

Colab link

In this part, we are going to run diffusion models on a larger scale. In particular, you will move from 28*28 grayscale FashionMNIST images to 256*256 RGB images. We will be using OpenAI's guided diffusion codebase. This repository contains a full-fledged implementation of multiple variants of diffusion models and pretrained score function checkpoints. We encourage you to play around with the repository if you have enough Colab credits. For this part, we will remove almost all the scaffolding in previous experiments:

- You need to clone the codebase, set up the environment, and read the given instructions to learn how to use it all on your own.
- There will not be any instructions on where you should add your code, and we will directly describe what we are looking for. You need to figure out where to add code to generate the outputs we ask for.
- You will have total freedom about how you want to code it and everything is just like when you do research!

Here are some tips about using this repository:

- The generated samples will be saved to a folder in `/tmp` by default. We suggest that you add an additional argument called `--save_dir` to the `argparser`, and change `logger.configure()` to `logger.configure(args.save_dir)` so that you can set which directory each run should save to.
- If you encounter "package not found error" when running the scripts, you can add the following two lines to the script you are running, which adds the directory to PYTHONPATH environment variable.

```
import os, sys
sys.path.append(os.getcwd())
```

Since the questions below might require modifying the same script in the repository in different ways, we do not require you to run everything again on Colab if you run these experiments locally. If you do need Colab to do this part, you can run in the given Colab notebook. No matter where you run these experiments, *you only need to include the code for plotting in the Colab notebook.* Your code can just load the `.npz` file generated automatically in `save_dir`, and plot the figure for each part as instructed. *The runtime for each of the subparts is 5-10 minutes with Google Colab Pro+.*

A. (3pts) First, let's try unconditional image generation. Clone the guided diffusion codebase to your local GPU-powered computer or your Google Drive, and download the `256x256_diffusion_uncond.pt` to `models/` folder under the repository. Generate 8 random images and show them together, as if you are showing an image of size 256 (height) * 2048 (width). Include both your figure and your command to generate it in this writeup. If your command includes the `SAMPLE_FLAGS` and `MODEL_FLAGS` variables, please also include them here. The point is to show us the parameters with which you generated your samples. Briefly comment on the generated images.



Very random and different images of varying classes. Very different perspectives shown.

Flags:

```
import os
# Example of setting a writable directory
SAVE_DIR="/content/drive/MyDrive/Photos/guided-diffusion/samples"

# Ensure the directory exists
if not os.path.exists(SAVE_DIR):
    os.makedirs(SAVE_DIR)

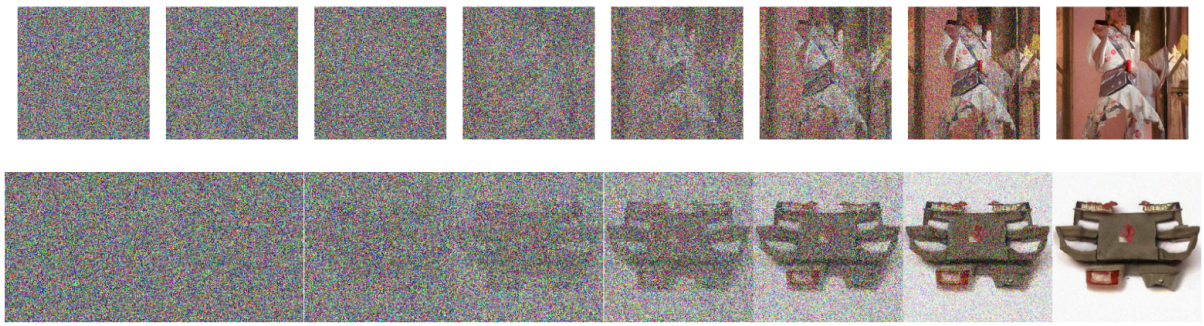
# Define flags
SAMPLE_FLAGS="""
    --batch_size 8
    --num_samples 8
    --timestep_respacing 250
"""
MODEL_FLAGS="""
    --attention_resolutions 32,16,8
    --class_cond False
    --diffusion_steps 1000
    --image_size 256
    --learn_sigma True
    --noise_schedule linear
    --num_channels 256
    --num_head_channels 64
    --num_res_blocks 2
    --resblock_updown True
    --use_fp16 True
```

```
   --use_scale_shift_norm True
"""

!python scripts/image_sample.py {MODEL_FLAGS.replace('\n', '')} \
   --model_path models/256x256_diffusion_uncond.pt \
   --save_dir /content/drive/MyDrive/Photos/guided-diffusion/samples \
   {SAMPLE_FLAGS.replace('\n', '')}
```

B. (3pts) Let's visualize the evolution of the image generation process. Please follow these instructions to generate your image:
   a. Your final image should be of size 256 (height) * 2048 (width), consisting of 8 images in eight columns.
   b. The initial random noises are in the first column and the final generated images are in the last column.
   c. The time steps of the middle columns are evenly spaced between the initial and final time.
   d. See Figure 14 in [4] for an example.

   Again, include both your figure and your command to generate it in this writeup. If your command includes the `SAMPLE_FLAGS` and `MODEL_FLAGS` variables, please also include them here. Comment on progressive generation process.



It shows a progression from a noisy to a clean image in a sequential manner.
Starts from pure noise and as time progresses the image becomes more and more clear as we are removing noise from the image at each time step.

Flags:

```
SAVE_DIR="/samples"

SAMPLE_FLAGS="""
   --batch_size 1
```

```
   --num_samples 8
   --timestep_respacing 250
"""
MODEL_FLAGS="""
   --attention_resolutions 32,16,8
   --class_cond False
   --diffusion_steps 1000
   --image_size 256
   --learn_sigma True
   --noise_schedule linear
   --num_channels 256
   --num_head_channels 64
   --num_res_blocks 2
   --resblock_updown True
   --use_fp16 True
   --use_scale_shift_norm True
"""

!python scripts/image_sample_evolution.py {MODEL_FLAGS.replace('\n',
'')} \
   --model_path models/256x256_diffusion_uncond.pt \
   --save_dir SAVE_DIR \
   {SAMPLE_FLAGS.replace('\n', '')}
```

I implemented my plotting in image_sample_evolution.py. Here is the function I used and this is how I called it:

```
def visualize_samples(samples, save_path):
    num_samples = len(samples)
    num_rows = (num_samples + 7) // 8
    fig_width = 2048 / 100
    fig_height = 256 * num_rows / 100
    fig = plt.figure(figsize=(fig_width, fig_height))

    for i, sample in enumerate(samples):
        sample = ((sample + 1) * 127.5).clamp(0,
255).to(th.uint8).cpu().numpy()
        sample = np.transpose(sample, (1, 2, 0))
        row_idx = i // 8
        col_idx = i % 8
        ax = fig.add_subplot(num_rows, 8, i+1)
        ax.imshow(sample)
        ax.axis('off')

    plt.subplots_adjust(wspace=0, hspace=0)
```

```
    plt.savefig(save_path, bbox_inches='tight', pad_inches=0)
    plt.close(fig)

save_path = os.path.join(logger.get_dir(), "sample_interpolation.png")
    visualize_samples(samples, save_path)
    logger.log(f"Saved visualization to {save_path}")
```

C. (2pts) Then, let's see a comparison of starting from different initial noise states along a trajectory. Generate two instances of standard Gaussian noise z_0 and z_7. Try to pick z_0 and z_7 such that their corresponding samples are as different as possible. Then, generate 8 samples with evenly spaced points between the two generated noise ($z_i = (1 - \frac{i}{7})z_0 + \frac{i}{7}z_7$ for $i = 0, \ldots, 7$). Please follow these instructions to generate your image:

   a. Your final image should be of size 256 (height) * 2048 (width), consisting of 8 images in eight columns.
   b. See Figure 6 in [4] for an example.
   c. Your figure should demonstrate a progression of interpolation. To do so, you need to fix the random seed for the generation process and let the initial state be the only variable.

   Again, include both your figure and your command to generate it in this writeup. If your command includes the `SAMPLE_FLAGS` and `MODEL_FLAGS` variables, please also include them here. Comment on the progressive generation process.



It goes from the first image to the last image by transforming into different images at each step. Each step in the image is changed a little bit from the previous one to make it into the new image. We are essentially adding noise step by step to the same image till it reaches the final image, and all the images in between are also "realistic" images.

Flags:

```
SAVE_DIR="/samples"
```

```
SAMPLE_FLAGS="""
   --batch_size 1
   --num_samples 8
   --timestep_respacing 250
"""
MODEL_FLAGS="""
   --attention_resolutions 32,16,8
   --class_cond False
   --diffusion_steps 1000
   --image_size 256
   --learn_sigma True
   --noise_schedule linear
   --num_channels 256
   --num_head_channels 64
   --num_res_blocks 2
   --resblock_updown True
   --use_fp16 True
   --use_scale_shift_norm True
"""

!python scripts/image_sample_c.py {MODEL_FLAGS.replace('\n', '')} \
   --model_path models/256x256_diffusion_uncond.pt \
   --save_dir SAVE_DIR \
   {SAMPLE_FLAGS.replace('\n', '')}
```

I used the same code for plotting as I mentioned for part b. Also the output in my colab notebook is wrong I just stored the corrected image to the directory. Let me know if you need to see my files for part b and part c

D. (2pts) Now, we explore the conditional image generation, which is the classifier-guided sampling mentioned in part 3G. Download the `256x256_classifier.pt` to `models/` folder under the repository. Generate 8 random images and show them together, as if you are showing an image of size 256 (height) * 2048 (width). These 8 images should be conditioned on 8 random classes. Include both your figure and your command to generate it in this writeup. If your command includes the `SAMPLE_FLAGS` and `MODEL_FLAGS` variables, please also include them here. Briefly comment on the generated images.



They are all clean images of very distinct classes. No class is repeated. So, we can clearly see that each image is conditioned on a different class. Each image is also taken from a different perspective.

Flags:

```
SAMPLE_FLAGS = """
   --batch_size 8
   --num_samples 8
   --timestep_respacing 250
"""
MODEL_FLAGS = """
   --attention_resolutions 32,16,8
   --class_cond False
   --diffusion_steps 1000
   --image_size 256
   --learn_sigma True
   --noise_schedule linear
   --num_channels 256
   --num_head_channels 64
   --num_res_blocks 2
   --resblock_updown True
   --use_fp16 True
   --use_scale_shift_norm True
"""

# run a script
!python scripts/classifier_sample.py {MODEL_FLAGS.replace('\n', '')} \
   --classifier_scale 10.0 \
   --classifier_path models/256x256_classifier.pt \
   --model_path models/256x256_diffusion_uncond.pt \
   {SAMPLE_FLAGS.replace('\n', '')}
```

E. (2pts) In the classifier-guided sampling case, there will be a hyperparameter (`--classifier_scale`) that controls the strength of the classifier gradient in the generation process. Pick a reasonable range for this parameter and plot the generated

samples corresponding to eight values within the range (not necessarily evenly spaced but monotonically increasing). Please follow these instructions to generate your image:

a. Your final image should be of size 512 (height) * 2048 (width), consisting of 16 images in two rows and eight columns.
b. Each column corresponds to one `classifier_scale` value, and the two rows are two different samples with the same `classifier_scale` value.
c. All 8 images should have the same initial state and generation process with the same random seed. The only changing variable is the value of the `classifier_scale` argument.
d. We expect to see a nice progression from classifiers kicking in the generation process. The lower bound of the range can be just 0. You can pick any reasonable upper bound.

Include both your figure and your command to generate it in this writeup. If your command includes the `SAMPLE_FLAGS` and `MODEL_FLAGS` variables, please also include them here. Briefly comment on the generated images.



With low --classifier_scale = 0:
No Classifier Guidance: The generated images are solely influenced by the generative model without any external guidance. These images lack coherence and are less recognizable or relevant to the target class. The diversity might be higher, but at the expense of quality and class-specific details.
With Low --classifier_scale Values (e.g., 1 to 2): Subtle Classifier Influence: The classifier begins to have a slight impact on the generation process. The images start showing weak features related to the target class. There is a balance between diversity and relevance to the class, but the details are still somewhat ambiguous.
With Medium --classifier_scale Values (e.g., 5 to 10): Stronger Classifier Guidance: The generated images exhibit more distinct and recognizable features of the target class. The details become sharper and more relevant. However, some variability is still present, providing a balance between coherence and diversity. The images are more refined and closer to what a human might expect for the target class.
With High --classifier_scale Values (e.g., 10 to 25):
Dominant Classifier Influence: The classifier's guidance significantly shapes the generation process. The images should be very clear and highly relevant to the target class, with well-defined features and minimal noise. However, too much classifier influence can sometimes lead to overfitting where the generated samples become less diverse and too similar to each other or overly reliant on the classifier's bias.

Potential Issues with Extremely High --classifier_scale Values: Overfitting and Reduced Diversity: If the classifier scale is too high, the generative process might produce almost identical images with little variation, reflecting the classifier's strong bias. This could lead to images that lack natural diversity and might miss some nuances or variations present in the target class.

Flags:

```
SAMPLE_FLAGS = """
   --batch_size 2
   --num_samples 2
   --timestep_respacing 250
"""
MODEL_FLAGS = """
   --attention_resolutions 32,16,8
   --class_cond False
   --diffusion_steps 1000
   --image_size 256
   --learn_sigma True
   --noise_schedule linear
   --num_channels 256
   --num_head_channels 64
   --num_res_blocks 2
   --resblock_updown True
   --use_fp16 True
   --use_scale_shift_norm True
"""
```

References

[1] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. ArXiv, abs/2006.11239.
[2] Sohl-Dickstein, J.N., Weiss, E.A., Maheswaranathan, N., & Ganguli, S. (2015). Deep Unsupervised Learning using Nonequilibrium Thermodynamics. ArXiv, abs/1503.03585.
[3] Vincent, P. (2011). A Connection Between Score Matching and Denoising Autoencoders. Neural Computation, 23, 1661-1674.
[4] Song, Y., Sohl-Dickstein, J.N., Kingma, D.P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. ICLR 2021.
[5] Anderson, B.D. (1982). Reverse-time diffusion equation models. Stochastic Processes and their Applications, 12, 313-326.